



.Net

Teoría 10

Una introducción rápida a LINQ

LINQ

En la práctica de delegados, se pidió extender al tipo `int[]` con los métodos `Seleccionar` y `Donde`. Por ejemplo si `v` es un vector de enteros, la expresión

```
v.Donde(n => n % 2 == 1).Seleccionar(n => n * n)
```

debía devolver un nuevo vector con todos los elementos impares de `v` elevados al cuadrado.

Los delegados como parámetros aportan muchísima versatilidad.



LINQ

Si en lugar de extender `int[]` extendiésemos `IEnumerable<T>` sería aún más beneficioso porque afectaría a todas las colecciones que implementan esta interfaz. Esto se pidió como ejercicio en la práctica de genéricos.

Afortunadamente no tenemos que hacerlo
`LINQ` ya lo hace por nosotros

Veamos algunos ejemplos ...





Crear una aplicación de consola llamada LINQ



1. Abrir una terminal del sistema operativo
2. Cambiar a la carpeta `proyectosDotnet`
3. Crear la aplicación de consola `LINQ`
4. Abrir code en la carpeta `LINQ`



Codificar Program.cs de la siguiente manera y ejecutar



```
int[] vector = [1, 2, 3, 4, 5];  
IEnumerable<int> secuencia = vector.Select(n => n * 3);  
Mostrar(secuencia);
```

```
void Mostrar<T>(IEnumerable<T> secuencia)  
{  
    foreach (T elemento in secuencia)  
    {  
        Console.Write(elemento + " - ");  
    }  
    Console.WriteLine();  
}
```

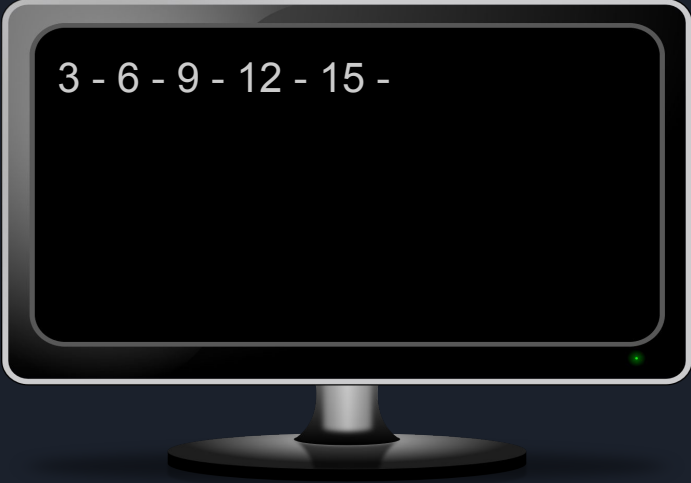
Copiar el código del archivo
10_RecursosParaLaTeoria.txt

```
int[] vector = [1, 2, 3, 4, 5];  
IEnumerable<int> secuencia = vector.Select(n => n * 3);  
Mostrar(secuencia);
```

T[] implementa la interfaz
IEnumerable<T>

```
void Mostrar<T>(IEnumerable<T> secuencia)  
{  
    foreach (T elemento in secuencia)  
    {  
        Console.Write(elemento + " ");  
    }  
    Console.WriteLine();  
}
```

Obtenemos los
elementos de **vector**
multiplicados por 3



3 - 6 - 9 - 12 - 15 -



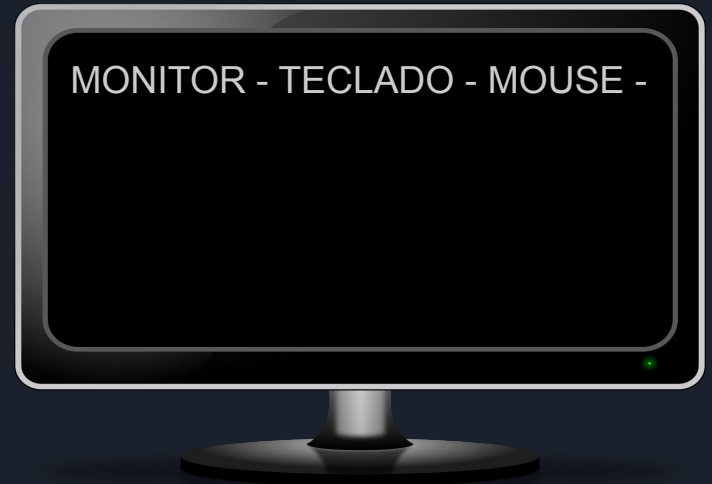
Completar la línea que falta para que la salida por consola sea la que se indica



XX

```
Mostrar(secuencia);
```

```
void Mostrar<T>(IEnumerable<T> secuencia)
{
    foreach (T elemento in secuencia)
    {
        Console.Write(elemento + " - ");
    }
    Console.WriteLine();
}
```

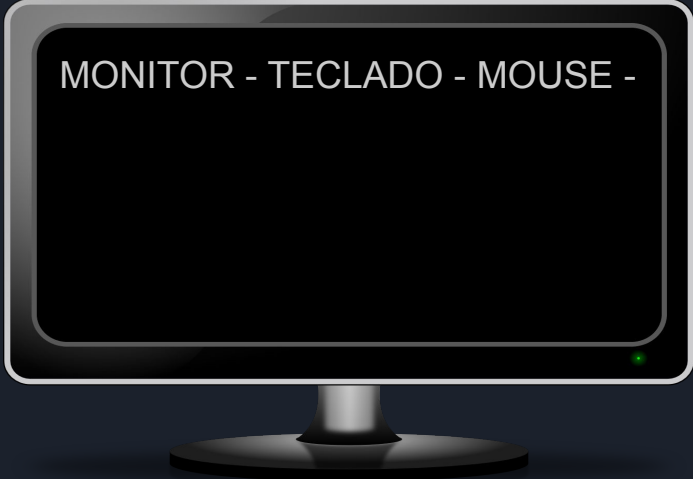


Copiar el código del archivo
10_RecursosParaLaTeoria.txt

Posible solución

```
List<string> lista = [" monitor ", " teclado", "mouse "];  
// DESAFÍO: Limpiar espacios en blanco y convertir a mayúsculas  
→ IEnumerable<string> secuencia = lista.Select(st => st.Trim().ToUpper());  
Mostrar(secuencia);
```

```
void Mostrar<T>(IEnumerable<T> secuencia)  
{  
    foreach (T elemento in secuencia)  
    {  
        Console.Write(elemento + " - ");  
    }  
    Console.WriteLine();  
}
```



MONITOR - TECLADO - MOUSE -



Completar la línea que falta para que la salida por consola sea la que se indica



```
List<string> lista = [ " monitor ", " teclado", "mouse "];  
IEnumerable<string> secuencia = lista.Select(st => st.Trim().ToUpper());  
Mostrar(secuencia);  
// Ahora queremos saber cuántas letras tiene cada nombre  
→ IEnumerable<int> secuencia2 = xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx  
Mostrar(secuencia2);
```

```
void Mostrar<T>(IEnumerable<T> secuencia)  
{  
    foreach (T elemento in secuencia)  
    {  
        Console.Write(elemento + " - ");  
    }  
    Console.WriteLine();  
}
```

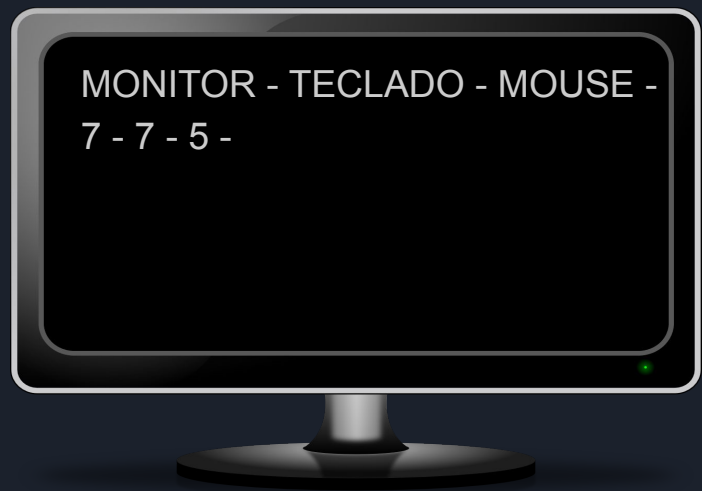
Longitud de los
strings de secuencia

MONITOR - TECLADO - MOUSE -
7 - 7 - 5 -

Posible solución

```
List<string> lista = [" monitor ", " teclado", "mouse "];  
IEnumerable<string> secuencia = lista.Select(st => st.Trim().ToUpper());  
Mostrar(secuencia);  
// Ahora queremos saber cuántas letras tiene cada nombre  
IEnumerable<int> secuencia2 = secuencia.Select(st => st.Length);  
Mostrar(secuencia2);
```

```
void Mostrar<T>(IEnumerable<T> secuencia)  
{  
    foreach (T elemento in secuencia)  
    {  
        Console.Write(elemento + " - ");  
    }  
    Console.WriteLine();  
}
```



MONITOR - TECLADO - MOUSE -
7 - 7 - 5 -



Posible solución

```
List<string> lista = [" monitor ", " teclado", "mouse "];  
IEnumerable<string> secuencia = lista.Select(st => st.Trim().ToUpper());  
Mostrar(secuencia);  
// Ahora queremos saber cuántas letras tiene cada nombre  
IEnumerable<int> secuencia2 = secuencia.Select(st => st.Length);  
Mostrar(secuencia2);
```



Observar que `secuencia2` es de un tipo distinto a `secuencia` (el método `Select` es un método genérico, se está haciendo inferencia de parámetros de tipos a partir del argumento, en este caso de tipo `Func<string,int>`), por lo tanto se está invocando a `Select<string,int>`





Posible solución

```
List<string> lista = [" monitor ", " teclado", "mouse "];  
IEnumerable<string> secuencia = lista.Select(st => st.Trim().ToUpper());  
Mostrar(secuencia);  
// Ahora queremos saber cuántas letras tiene cada nombre  
IEnumerable<int> secuencia2 = secuencia.Select(st => st.Length);  
Mostrar(secuencia2);
```

Este es el encabezado del método de extensión `Select` definido en la clase estática `System.Linq.Enumerable`

```
public static IEnumerable<TResult> Select<TSource, TResult>(  
    this IEnumerable<TSource> source, Func<TSource, TResult> selector)
```



Completar la línea que falta para que la salida por consola sea la que se indica



```
List<string> lista = [" monitor ", " teclado", "mouse "];  
IEnumerable<string> secuencia = lista.Select(st => st.Trim().ToUpper());  
Mostrar(secuencia);  
IEnumerable<int> secuencia2 = secuencia.Select(st => st.Length);  
Mostrar(secuencia2);  
// Calcularemos el precio de impresión de cada palabra  
// como la mitad de su longitud  
→ IEnumerable<double> secuencia3 = xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx  
Mostrar(secuencia3);
```

```
void Mostrar<T>(IEnumerable<T> secuencia)  
{  
    foreach (T elemento in secuencia)  
    {  
        Console.Write(elemento + " - ");  
    }  
    Console.WriteLine();  
}
```

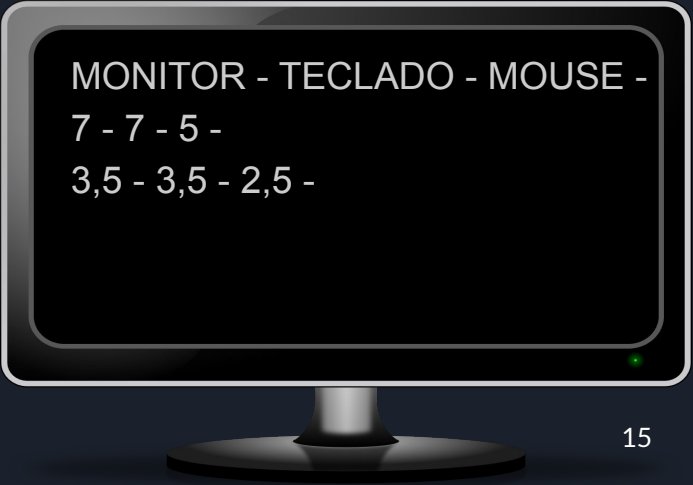
MONITOR - TECLADO - MOUSE -
7 - 7 - 5 -
3,5 - 3,5 - 2,5 -



Posible solución

```
List<string> lista = [" monitor ", " teclado", "mouse "];  
IEnumerable<string> secuencia = lista.Select(st => st.Trim().ToUpper());  
Mostrar(secuencia);  
IEnumerable<int> secuencia2 = secuencia.Select(st => st.Length);  
Mostrar(secuencia2);  
// Calculamos el precio de impresión de cada palabra  
// como la mitad de su longitud  
IEnumerable<double> secuencia3 = secuencia2.Select(n => n / 2.0);  
Mostrar(secuencia3);
```

```
void Mostrar<T>(IEnumerable<T> secuencia)  
{  
    foreach (T elemento in secuencia)  
    {  
        Console.Write(elemento + " - ");  
    }  
    Console.WriteLine();  
}
```



MONITOR - TECLADO - MOUSE -
7 - 7 - 5 -
3,5 - 3,5 - 2,5 -



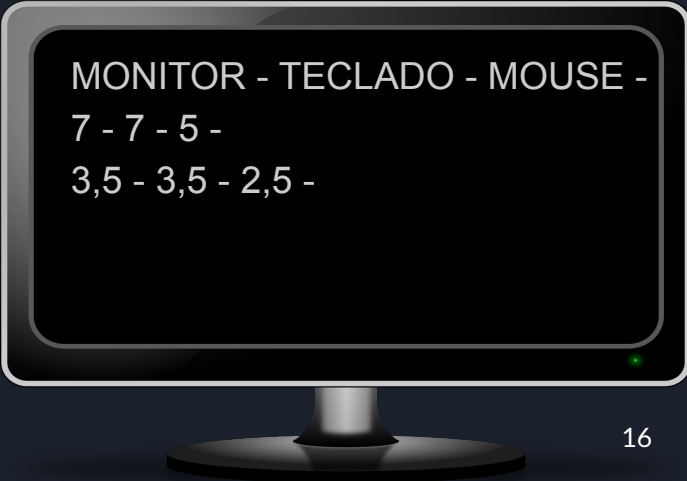
Posible solución

```
List<string> lista = [" monitor ", " teclado", "mouse "];  
IEnumerable<string> secuencia = lista.Select(st => st.Trim().ToUpper());  
Mostrar(secuencia);  
IEnumerable<int> secuencia2 = secuencia.Select(st => st.Length);  
Mostrar(secuencia2);  
// Calcularemos el precio de impresión de cada palabra  
// como la mitad de su longitud  
IEnumerable<double> secuencia3 = secuencia2.Select(n => n / 2.0);  
Mostrar(secuencia3);
```



Dividimos por 2.0 los elementos enteros de secuencia2 obteniendo un `IEnumerable<double>`

Los argumentos de tipo del método `Select` se infieren por medio del argumento que en este caso es de tipo `Func<int,double>`



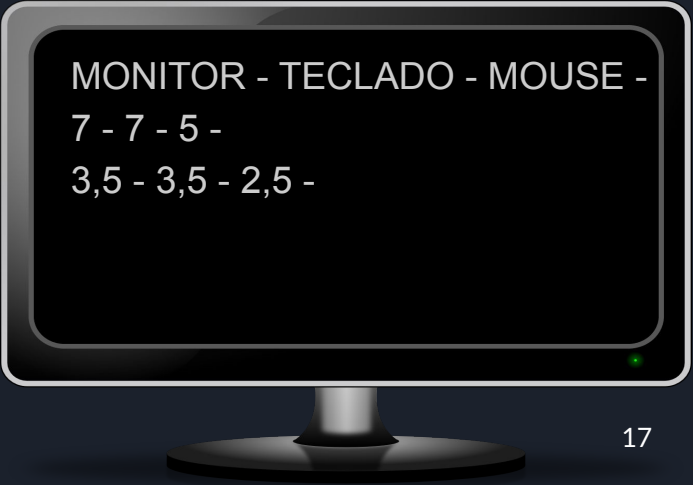
```
MONITOR - TECLADO - MOUSE -  
7 - 7 - 5 -  
3,5 - 3,5 - 2,5 -
```




Posible solución

```
List<string> lista = [" monitor ", " teclado", "mouse "];  
var secuencia = lista.Select(st => st.Trim().ToUpper());  
Mostrar(secuencia);  
var secuencia2 = secuencia.Select(st => st.Length);  
Mostrar(secuencia2);  
var secuencia3 = secuencia2.Select(n => n / 2.0);  
Mostrar(secuencia3);
```

Es muy común utilizar LINQ con inferencia de tipos (palabra clave `var`) para simplificar la escritura y lectura del código

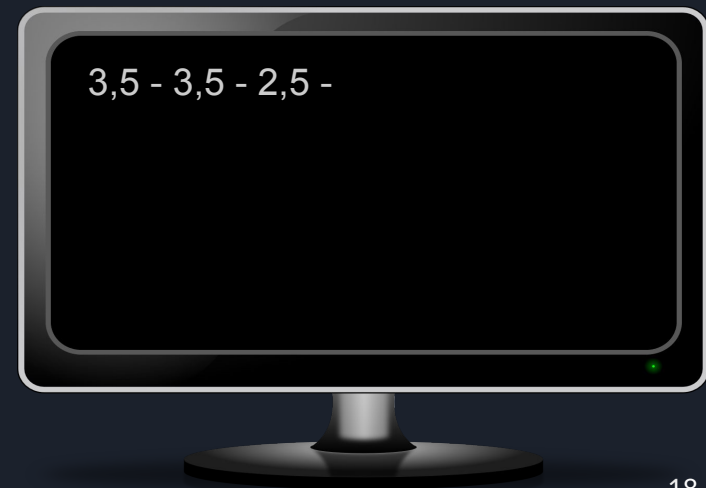


MONITOR - TECLADO - MOUSE -
7 - 7 - 5 -
3,5 - 3,5 - 2,5 -

Interfaz Fluida de LINQ

```
List<string> lista = [" monitor ", " teclado", "mouse "];  
var secuencia = lista.Select(st => st.Trim().ToUpper())  
                    .Select(st => st.Length)  
                    .Select(n => n / 2.0);  
Mostrar(secuencia);
```

Si sólo nos interesa el último resultado es muy común utilizar la interfaz fluida ([fluent API](#)) de [LINQ](#)



```
List<int> numeros = [1, 10, 7, 3, 11];  
Mostrar(numeros);
```

Además de `Select`, LINQ
provee muchos otros
métodos de extensión:
`Where`, `Reverse`, `OrderBy`,
`Sum`, `Average` son
sólo alguno de ellos



1 - 10 - 7 - 3 - 11 -

```
List<int> numeros = [1, 10, 7, 3, 11];  
Mostrar(numeros);  
var mayores6 = numeros.Where(n => n > 6);  
Mostrar(mayores6);
```

Además de Select, LINQ
provee muchos otros
métodos de extensión:
Where, Reverse, OrderBy,
Sum, Average son
sólo alguno de ellos



1 - 10 - 7 - 3 - 11 -
10 - 7 - 11 -

```
List<int> numeros = [1, 10, 7, 3, 11];  
Mostrar(numeros);  
var mayores6 = numeros.Where(n => n > 6);  
Mostrar(mayores6);  
var reverso = mayores6.Reverse();  
Mostrar(reverso);
```

Además de Select, LINQ
provee muchos otros
métodos de extensión:
Where, Reverse, OrderBy,
Sum, Average son
sólo alguno de ellos



1 - 10 - 7 - 3 - 11 -
10 - 7 - 11 -
11 - 7 - 10 -

```
List<int> numeros = [1, 10, 7, 3, 11];  
Mostrar(numeros);  
var mayores6 = numeros.Where(n => n > 6);  
Mostrar(mayores6);  
var reverso = mayores6.Reverse();  
Mostrar(reverso);  
var ordenados = reverso.OrderBy(n => n);  
Mostrar(ordenados);
```

Además de Select, LINQ
provee muchos otros
métodos de extensión:
Where, Reverse, OrderBy,
Sum, Average son
sólo alguno de ellos



1 - 10 - 7 - 3 - 11 -
10 - 7 - 11 -
11 - 7 - 10 -
7 - 10 - 11 -

```
List<int> numeros = [1, 10, 7, 3, 11];  
Mostrar(numeros);  
var mayores6 = numeros.Where(n => n > 6);  
Mostrar(mayores6);  
var reverso = mayores6.Reverse();  
Mostrar(reverso);  
var ordenados = reverso.OrderBy(n => n);  
Mostrar(ordenados);  
var suma = ordenados.Sum();  
var promedio = ordenados.Average();  
Console.WriteLine($"suma: {suma} promedio:{promedio:0.00}");
```

Además de Select, LINQ
provee muchos otros
métodos de extensión:
Where, Reverse, OrderBy,
Sum, Average son
sólo alguno de ellos



1 - 10 - 7 - 3 - 11 -
10 - 7 - 11 -
11 - 7 - 10 -
7 - 10 - 11 -

suma: 28 promedio:9,33

LINQ - Ejemplos

```
List<int> lista1 = [1,2,3,4,5,1,2,3,4,5];  
List<int> lista2 = [4,5,6,4,5,6];  
var concat = lista1.Concat(lista2);  
Mostrar(concat);
```

Otros métodos involucran
a más de una secuencia,
Concat, Union, Intersect y
Zip son
sólo alguno de ellos



1 - 2 - 3 - 4 - 5 - 1 - 2 - 3 - 4 - 5 - 4 - 5 - 6 - 4 - 5 - 6 -

LINQ - Ejemplos

```
List<int> lista1 = [1,2,3,4,5,1,2,3,4,5];  
List<int> lista2 = [4,5,6,4,5,6];  
var concat = lista1.Concat(lista2);  
Mostrar(concat);  
var union = lista1.Union(lista2);  
Mostrar(union);
```

Otros métodos involucran
a más de una secuencia,
Concat, Union, Intersect y
Zip son
sólo alguno de ellos



Representa la unión
entre conjuntos, por eso
no aparecen elementos
repetidos

1 - 2 - 3 - 4 - 5 - 1 - 2 - 3 - 4 - 5 - 4 - 5 - 6 - 4 - 5 - 6 -
1 - 2 - 3 - 4 - 5 - 6 -

LINQ - Ejemplos

```
List<int> lista1 = [1,2,3,4,5,1,2,3,4,5];  
List<int> lista2 = [4,5,6,4,5,6];  
var concat = lista1.Concat(lista2);  
Mostrar(concat);  
var union = lista1.Union(lista2);  
Mostrar(union);  
var interseccion = lista1.Intersect(lista2);  
Mostrar(interseccion);
```

Representa la
intersección entre
conjuntos, por eso no
aparecen elementos
repetidos

Otros métodos involucran
a más de una secuencia,
Concat, Union, Intersect y
Zip son
sólo alguno de ellos



1 - 2 - 3 - 4 - 5 - 1 - 2 - 3 - 4 - 5 - 4 - 5 - 6 - 4 - 5 - 6 -
1 - 2 - 3 - 4 - 5 - 6 -
4 - 5 -

LINQ - Ejemplos

```
List<int> lista1 = [1,2,3,4,5,1,2,3,4,5];  
List<int> lista2 = [4,5,6,4,5,6];  
var concat = lista1.Concat(lista2);  
Mostrar(concat);  
var union = lista1.Union(lista2);  
Mostrar(union);  
var interseccion = lista1.Intersect(lista2);  
Mostrar(interseccion);  
var zip = lista1.Zip(lista2, (a, b) => a + b);  
Mostrar(zip);
```

Otros métodos involucran
a más de una secuencia,
Concat, Union, Intersect y
Zip son
sólo alguno de ellos



Se utiliza para combinar dos
secuencias en una sola
secuencia aplicando una
función de combinación que se
aplica elemento a elemento
posicionalmente

+ [1,2,3,4, 5,1,2,3,4,5];
[4,5,6,4, 5,6];

[5,7,9,8,10,7]

1 - 2 - 3 - 4 - 5 - 1 - 2 - 3 - 4 - 5 - 4 - 5 - 6 - 4 - 5 - 6 -
1 - 2 - 3 - 4 - 5 - 6 -
4 - 5 -
5 - 7 - 9 - 8 - 10 - 7 -

//calculamos la suma de los primeros
//20 cuadrados: 1 + 4 + 9 + ... + 400

```
var suma = Enumerable.Range(1, 20)           // 1, 2, 3, ..., 20
                        .Select(n => n * n)    // 1, 4, 9, ... 400
                        .Sum();                // 1 + 4 + 9 + ... + 400
Console.WriteLine($"Resultado: {suma}");
```

Primer valor del rango

Cantidad de elementos
del rango



El método estático
Range(start,count) de la
clase System.Linq.Enumerable
devuelve un IEnumerable<int>
con la secuencia de enteros
comenzando por start
y con count elementos

Resultado: 2870



Codificar la clase
Persona que vamos a
utilizar para mostrar
más ejemplos de las
facilidades que brinda
LINQ

```
class Persona
{
    public string Nombre { get; private set; }
    public int Edad { get; private set; }
    public string Pais { get; private set; }
    public Persona(string nombre, int edad, string pais)
    {
        Nombre = nombre;
        Edad = edad;
        Pais = pais;
    }
    public override string ToString()
    {
        return $"{Nombre} ({Edad} años) {Pais.Substring(0, 3)}.";
    }

    // vamos a hardcodear una lista de personas
    // que usaremos en los siguientes ejemplos
    // para ello definimos el siguiente método estático
    public static List<Persona> GetLista()
    {
        return [
            new Persona("Pablo", 15, "Argentina"),
            new Persona("Juan", 55, "Argentina"),
            new Persona("José", 9, "Uruguay"),
            new Persona("María", 33, "Uruguay"),
            new Persona("Lucía", 16, "Perú"),
        ];
    }
}
```

Copiar el código del archivo
10_RecursosParaLaTeoria.txt



Completar el código para que la salida por consola sea la indicada (mayores de 18)



```
var personas = Persona.GetLista();  
personas.ForEach(p => Console.WriteLine(p)); // lista todas las personas  
Console.WriteLine();  
. . .
```

Listar las personas
mayores de edad

Pablo (15 años) Arg.
Juan (55 años) Arg.
José (9 años) Uru.
María (33 años) Uru.
Lucía (16 años) Per.

[Juan (55 años) Arg.
[María (33 años) Uru.

Copiar el código del archivo
10_RecursosParaLaTeoria.txt



Posible solución

```
var personas = Persona.GetLista();  
personas.ForEach(p => Console.WriteLine(p)); // lista todas las personas  
Console.WriteLine();  
personas.Where(p => p.Edad >= 18) // un IEnumerable<Persona> no tiene método Foreach  
    .ToList() // lo convierte en un List<Persona> para usar método Foreach  
    .ForEach(p => Console.WriteLine(p)); // lista personas mayores de edad
```

Pablo (15 años) Arg.
Juan (55 años) Arg.
José (9 años) Uru.
María (33 años) Uru.
Lucía (16 años) Per.

Juan (55 años) Arg.
María (33 años) Uru.

Para pensar: ¿Cómo deberíamos codificar el método de extensión `ImprimirEnConsola`?

```
Persona.GetLista()  
    .ImprimirEnConsola("Listado Completo")  
    .Where(p => p.Edad >= 18)  
    .ImprimirEnConsola("\nMayores de de edad");
```

LISTADO COMPLETO

- * Pablo (15 años) Arg.
- * Juan (55 años) Arg.
- * José (9 años) Uru.
- * María (33 años) Uru.
- * Lucía (16 años) Per.

MAYORES DE DE EDAD

- * Juan (55 años) Arg.
- * María (33 años) Uru.

Para pensar: ¿Cómo deberíamos codificar el método de extensión ImprimirEnConsola?

```
static class Extension
{
    public static IEnumerable<T> ImprimirEnConsola<T>(
        this IEnumerable<T> secuencia,
        string titulo)
    {
        Console.WriteLine(titulo.ToUpper());
        foreach (T t in secuencia)
        {
            Console.WriteLine($" * {t}");
        }
        return secuencia;
    }
}
```



```
Persona.GetLista()  
    .OrderBy(p => p.Edad)  
    .Select(p => new { Nombre = p.Nombre, Condición = p.Edad < 18 ? "Menor" : "Mayor" })  
    .ImprimirEnConsola("Usando tipos anónimos");
```



Aquí estamos
devolviendo un tipo
anónimos.
En este caso el método
Select devuelve un
IEnumerable de un tipo
anónimo que tiene las
propiedades
Nombre y Condición

USANDO TIPOS ANÓNIMOS

- * { Nombre = José, Condición = Menor }
- * { Nombre = Pablo, Condición = Menor }
- * { Nombre = Lucía, Condición = Menor }
- * { Nombre = María, Condición = Mayor }
- * { Nombre = Juan, Condición = Mayor }

LINQ - Ejemplos

```
var personas = Persona.GetLista();
Console.WriteLine($"Primero: {personas.First()}");
Console.WriteLine($"Primer mayor: {personas.First(p => p.Edad >= 18)}");
Console.WriteLine($"Último: {personas.Last()}");
Console.WriteLine($"Último mayor: {personas.Last(p => p.Edad >= 18)}");
Console.WriteLine($"Edad máxima: {personas.Max(p => p.Edad)}");
Console.WriteLine($"Persona con más edad: {personas.MaxBy(p => p.Edad)}");
Console.WriteLine($"Edad mínima: {personas.Min(p => p.Edad)}");
Console.WriteLine($"Son todos mayores? {personas.All(p => p.Edad >= 18)}");
Console.WriteLine($"Hay algún mayor? {personas.Any(p => p.Edad >= 18)}");
```

First, Last, Max,
MaxBy Min, MinBy All,
Any son algunos
métodos más que
brinda LINQ

```
...
//método en la clase Persona
public static List<Persona> GetLista()
{
    return [
        new Persona("Pablo",15,"Argentina"),
        new Persona("Juan", 55,"Argentina"),
        new Persona("José",9,"Uruguay"),
        new Persona("María",33,"Uruguay"),
        new Persona("Lucía",16,"Perú"),
    ];
}
...
```

Primero: Pablo (15 años) Arg.
Primer mayor: Juan (55 años) Arg.
Último: Lucía (16 años) Per.
Último mayor: María (33 años) Uru.
Edad máxima: 55
Persona con más edad: Juan (55 años) Arg.
Edad mínima: 9
Son todos mayores? False
Hay algún mayor? True

```
var personas = Persona.GetLista();

// =====
// CASO 1: First() vs FirstOrDefault()
// =====

// Buscamos una condición que NO se cumple (alguien de 100 años o más).
// la siguiente línea provocará una excepción: InvalidOperationException
Console.WriteLine($"Centenario: {personas.First(p => p.Edad >= 100)}");
```

Exception has occurred: CLR/System.InvalidOperationException ×
An unhandled exception of type 'System.InvalidOperationException' occurred in System.Linq.dll: 'Sequence contains no matching element'
at System.Linq.ThrowHelper.ThrowNoMatchException()
at System.Linq.Enumerable.First[TSource](IEnumerable`1 source, Func`2 predicate)
at Program.<Main>\$(String[] args) in /home/pc/mio/proyectos2026/LINQ/Program.cs:line 12

```
var personas = Persona.GetLista();

// Solución: FirstOrDefault()
// Si no encuentra coincidencias, en lugar de fallar, devuelve el valor por defecto del tipo.
// Como 'Persona' es una clase (tipo de referencia), el valor por defecto es 'null'.
var centenario = personas.FirstOrDefault(p => p.Edad >= 100);

if (centenario == null)
{
    Console.WriteLine("First() hubiera fallado. FirstOrDefault() devolvió null.");
}
else
{
    Console.WriteLine(centenario);
}
```



First() hubiera fallado. FirstOrDefault() devolvió null.

```
var personas = Persona.GetLista();

// =====
// CASO 2: Max() en secuencias vacías y el uso de DefaultIfEmpty()
// =====

// Creamos una secuencia vacía a propósito (no hay nadie de Chile en nuestra lista)
var personasDeChile = personas.Where(p => p.Pais == "Chile");

// La siguiente línea provocará una excepción: InvalidOperationException
Console.WriteLine($"Edad máxima en Chile: {personasDeChile.Max(p => p.Edad)}");
```

Exception has occurred: CLR/System.InvalidOperationException ✕

An unhandled exception of type 'System.InvalidOperationException' occurred in System.Linq.dll: 'Sequence contains no elements'

at System.Linq.ThrowHelper.ThrowNoElementsException()

at System.Linq.Enumerable.MaxInteger[TSource,TResult](IEnumerable`1 source, Func`2 selector)

at Program.<Main>\$(String[] args) in /home/pc/mio/proyectos2026/LINQ/Program.cs:line 14

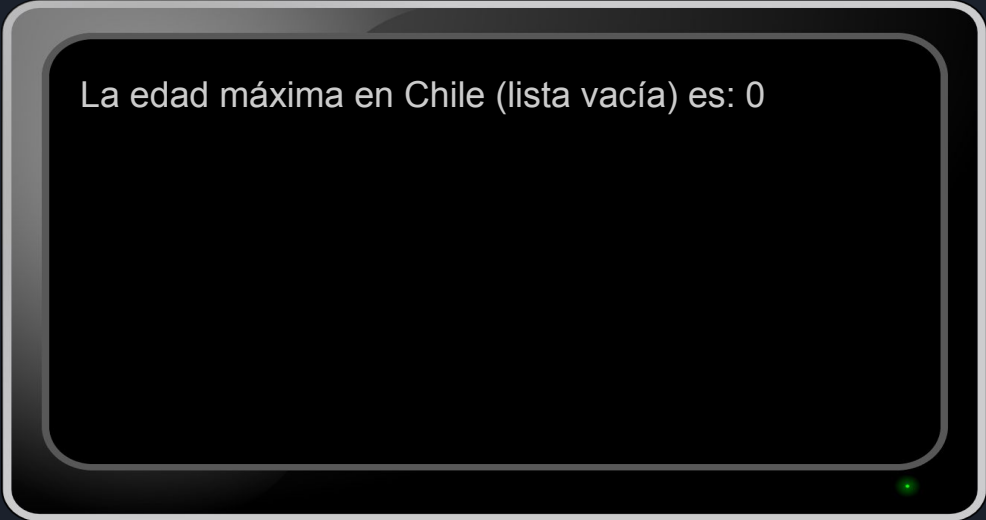
```
var personas = Persona.GetLista();

// Solución: Usar DefaultIfEmpty()

var personasDeChile = personas.Where(p => p.Pais == "Chile");

var edadMaximaSegura = personasDeChile
    .DefaultIfEmpty() // Al ver que la secuencia está vacía, genera una nueva con un
                      // con un único elemento, el predeterminado Persona (un null)
    .Max(p => p?.Edad); // Al usar '?' se infiere que edadMaximaSegura es de tipo int?

// Utilizamos el operador '??' para imprimir un 0 en caso de que el resultado sea 'null'.
Console.WriteLine($"La edad máxima en Chile (lista vacía) es: {edadMaximaSegura ?? 0}");
```



La edad máxima en Chile (lista vacía) es: 0

```
var personas = Persona.GetLista();
var grupos = personas.GroupBy(p => p.Pais);
foreach (var grup in grupos)
{
    Console.WriteLine($"{grup.Key} ({grup.Count()})");
    foreach(var p in grup )
    {
        Console.WriteLine(" * " + p);
    }
}
```

Agrupamos por país.
grupos es un IEnumerable de
IGrouping<string,Persona>

Cada grup representa un
grupo de personas junto a
su clave de agrupación.
grup es de tipo
IGrouping<string,Persona>,
esta interfaz hereda de
IEnumerable<Persona>



LINQ Nos permite
agrupar fácilmente
por algún criterio

```
Argentina (2)
* Pablo (15 años) Arg.
* Juan (55 años) Arg.
Uruguay (2)
* José (9 años) Uru.
* María (33 años) Uru.
Perú (1)
* Lucía (16 años) Per.
```



```
Persona.GetLista()  
    .GroupBy(p => p.Pais)  
    .ToList()  
    .ForEach(grup =>  
    {  
        Console.WriteLine($"{grup.Key} ({grup.Count()})");  
        grup.ToList().ForEach(p => Console.WriteLine(" * " + p));  
    });
```

También podemos
hacerlo de esta manera
evitando la estructura
de control `foreach`.



```
Argentina (2)  
 * Pablo (15 años) Arg.  
 * Juan (55 años) Arg.  
Uruguay (2)  
 * José (9 años) Uru.  
 * María (33 años) Uru.  
Perú (1)  
 * Lucía (16 años) Per.
```

```
Persona.GetLista()  
    .GroupBy(p => p.Pais)  
    .ToList()  
    .ForEach(grup => grup.ImprimirEnConsola($"{grup.Key} ({grup.Count()})"));
```



Utilizando nuestro
método
ImprimirEnConsola
(100% interfaz fluida)

A computer monitor with a black bezel and a silver stand. The screen displays the output of the LINQ query, showing groups of people by country. A small green light is visible at the bottom right of the monitor.

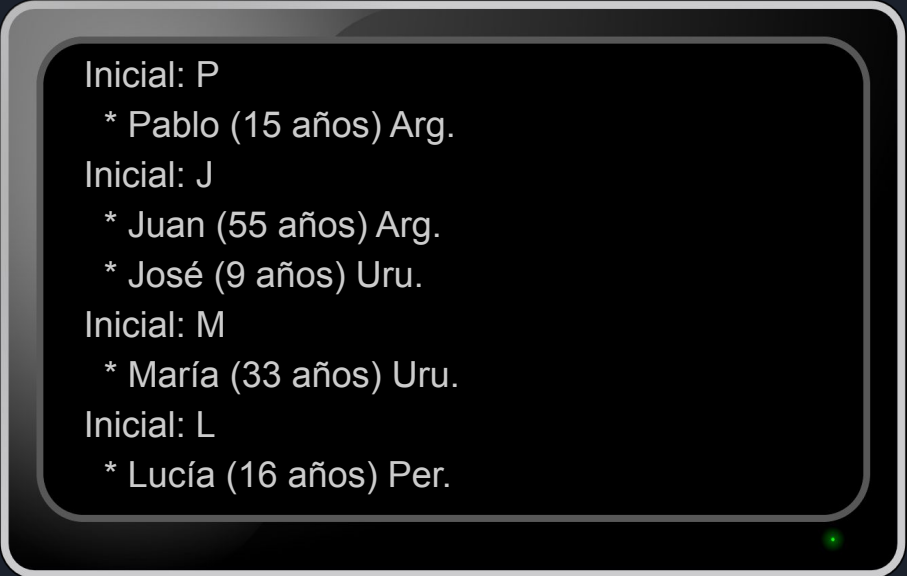
```
Argentina (2)  
* Pablo (15 años) Arg.  
* Juan (55 años) Arg.  
Uruguay (2)  
* José (9 años) Uru.  
* María (33 años) Uru.  
Perú (1)  
* Lucía (16 años) Per.
```



Utilizar GroupBy para listar las personas agrupadas por la inicial de su nombre



```
var personas = Persona.GetLista();  
var agrupadas = personas.GroupBy(xxxxxxxxxxxxxxxxxx);  
foreach(var grupo in agrupadas)  
    . . .
```

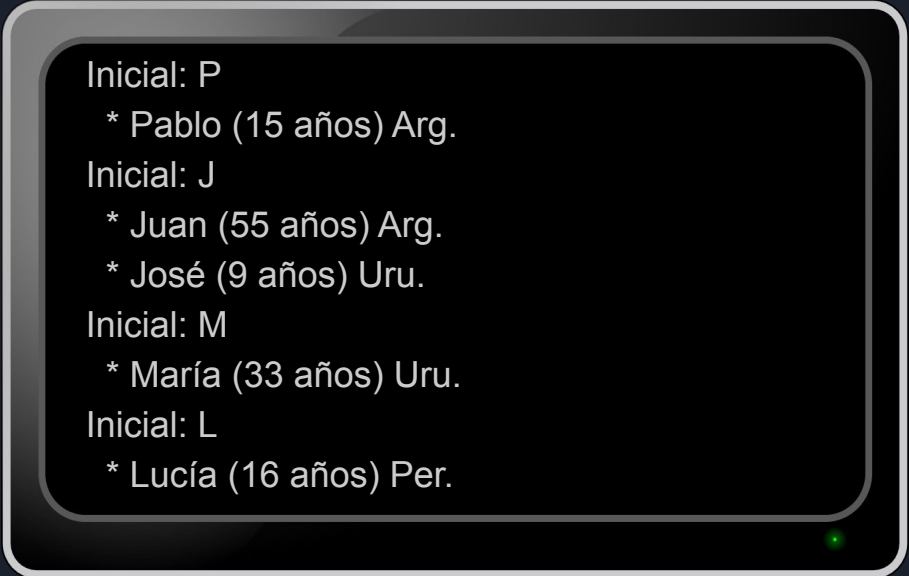


```
Inicial: P  
    * Pablo (15 años) Arg.  
Inicial: J  
    * Juan (55 años) Arg.  
    * José (9 años) Uru.  
Inicial: M  
    * María (33 años) Uru.  
Inicial: L  
    * Lucía (16 años) Per.
```



Posible solución

```
var personas = Persona.GetLista();  
var agrupadas = personas.GroupBy(p => p.Nombre[0]);  
foreach(var grupo in agrupadas)  
{  
    Console.WriteLine($"Inicial: {grupo.Key}");  
    grupo.ToList().ForEach(p => Console.WriteLine("    * " + p));  
}
```



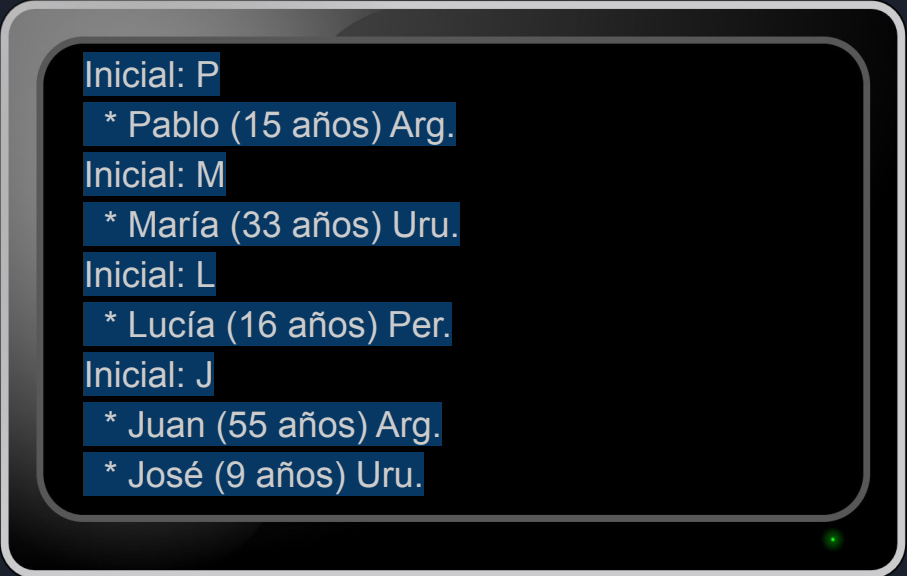
```
Inicial: P  
    * Pablo (15 años) Arg.  
Inicial: J  
    * Juan (55 años) Arg.  
    * José (9 años) Uru.  
Inicial: M  
    * María (33 años) Uru.  
Inicial: L  
    * Lucía (16 años) Per.
```



Ordenar el listado por inicial de manera descendente



```
var personas = Persona.GetLista();  
var agrupadas = personas.GroupBy(p => p.Nombre[0]).xxxxxxxxxxxxxxxxxx;  
foreach(var grupo in agrupadas)  
{  
    Console.WriteLine($"Inicial: {grupo.Key}");  
    grupo.ToList().ForEach(p => Console.WriteLine(" * " + p));  
}
```



```
Inicial: P  
 * Pablo (15 años) Arg.  
Inicial: M  
 * María (33 años) Uru.  
Inicial: L  
 * Lucía (16 años) Per.  
Inicial: J  
 * Juan (55 años) Arg.  
 * José (9 años) Uru.
```



Posible solución

```
var personas = Persona.GetLista();
var agrupadas = personas.GroupBy(p => p.Nombre[0]).OrderByDescending(g=>g.Key);
foreach(var grupo in agrupadas)
{
    Console.WriteLine($"Inicial: {grupo.Key}");
    grupo.ToList().ForEach(p => Console.WriteLine(" * " + p));
}
```

A computer monitor with a black bezel and a silver stand, displaying the output of the LINQ query. The text on the screen is as follows:

Inicial: P
* Pablo (15 años) Arg.
Inicial: M
* María (33 años) Uru.
Inicial: L
* Lucía (16 años) Per.
Inicial: J
* Juan (55 años) Arg.
* José (9 años) Uru.



Posible solución

```
Persona.GetLista()  
    .GroupBy(p => p.Nombre[0])  
    .OrderByDescending(g => g.Key)  
    .ToList()  
    .ForEach(grupo => grupo.ImprimirEnConsola($"Inicial: {grupo.Key}"));
```



Utilizando interfaz
fluida y nuestro
método
ImprimirEnConsola

Inicial: P
* Pablo (15 años) Arg.
Inicial: M
* María (33 años) Uru.
Inicial: L
* Lucía (16 años) Per.
Inicial: J
* Juan (55 años) Arg.
* José (9 años) Uru.



Ejercicio



Utilizando el método `Enumerable.Range` agrupar los números enteros entre 1 y 20 según el resto de dividir por 3

```
Resto de dividir por 3 = 1
```

```
1 4 7 10 13 16 19
```

```
Resto de dividir por 3 = 2
```

```
2 5 8 11 14 17 20
```

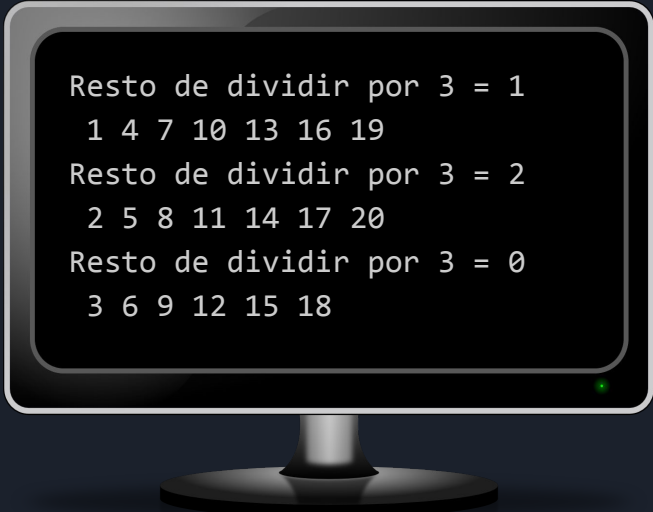
```
Resto de dividir por 3 = 0
```

```
3 6 9 12 15 18
```




Posible Solución

```
Enumerable.Range(1, 20)
    .GroupBy(i => i % 3)
    .ToList()
    .ForEach(grupo =>
    {
        Console.WriteLine($"Resto de dividir por 3 = {grupo.Key}");
        grupo.ToList().ForEach(i => Console.Write($" {i}"));
        Console.WriteLine();
    });
```



```
Resto de dividir por 3 = 1
 1 4 7 10 13 16 19
Resto de dividir por 3 = 2
 2 5 8 11 14 17 20
Resto de dividir por 3 = 0
 3 6 9 12 15 18
```



Vamos a utilizar
estas dos clases
Alumno y
Examen en los
siguientes
ejemplos

```
class Alumno
{
    public int Id { get; private set; }
    public string Nombre { get; private set; }
    public Alumno(int id, string nombre)
    {
        Id = id;
        Nombre = nombre;
    }
}

class Examen
{
    public int AlumnoId { get; private set; }
    public string Materia { get; private set; }
    public double Nota { get; private set; }
    public Examen(int alumnoId, string materia, double nota)
    {
        AlumnoId = alumnoId;
        Materia = materia;
        Nota = nota;
    }
}
```

```
var alumnos = new List<Alumno>() {  
    new Alumno(1,"Juan"),  
    new Alumno(2,"Ana"),  
    new Alumno(3,"Laura")  
};  
  
var examenes = new List<Examen>() {  
    new Examen(2,"Inglés",9),  
    new Examen(1,"Inglés",5),  
    new Examen(1,"Álgebra",10)  
};  
  
var listado = alumnos.Join(examenes,  
    a => a.Id, //clave de matching en alumnos  
    e => e.AlumnoId, //clave de matching en exámenes  
    (a, e) => new  
    {  
        Alumno = a.Nombre,  
        Materia = e.Materia,  
        Nota = e.Nota  
    });  
  
listado.ToList().ForEach(obj => Console.WriteLine(obj));
```

Alumnos

Nombre	Id
Juan	1
Ana	2
Laura	3

Exámenes

AlumnoId	Materia	Nota
2	Inglés	9
1	Inglés	5
1	Álgebra	10

Join es como inner join de SQL, devuelve sólo los elementos donde las claves coinciden.

```
{ Alumno = Juan, Materia = Inglés, Nota = 5 }  
{ Alumno = Juan, Materia = Álgebra, Nota = 10 }  
{ Alumno = Ana, Materia = Inglés, Nota = 9 }
```

LINQ - Ejemplos

```
var alumnos = new List<Alumno>() {  
    new Alumno(1,"Juan"),  
    new Alumno(2,"Ana"),  
    new Alumno(3,"Laura")  
};
```

```
var examenes = new List<Examen>() {  
    new Examen(2,"Inglés",9),  
    new Examen(1,"Inglés",5),  
    new Examen(1,"Álgebra",10)  
};
```

```
var listado = alumnos.GroupJoin(examenes,  
    a => a.Id, //clave de matching en alumnos  
    e => e.AlumnoId, //clave de matching en examenes  
    (a, susExamenes) => new  
    {  
        NombreAlumno = a.Nombre,  
        Examenes = susExamenes  
    });
```

```
foreach (var grup in listado)  
{  
    Console.WriteLine($"Alumno: {grup.NombreAlumno}");  
    Console.WriteLine($" Cant. exams.: {grup.Examenes.Count()}");  
    foreach (var e in grup.Examenes)  
    {  
        Console.WriteLine($" {e.Materia} Nota:{e.Nota}");  
    }  
}
```

Alumnos

Nombre	Id
Juan	1
Ana	2
Laura	3

Listar a todos los alumnos,
incluso los que no hayan
rendido ningún examen

Exámenes

AlumnoId	Materia	Nota
2	Inglés	9
1	Inglés	5
1	Álgebra	10

Can GroupJoin
agrupamos a los
alumno con sus
exámenes. También
obtengo información
de quienes no rindieron
ninguno

```
Alumno: Juan, Cant. exams.: 2  
Inglés Nota:5  
Álgebra Nota:10  
Alumno: Ana, Cant. exams.: 1  
Inglés Nota:9  
Alumno: Laura, Cant. exams.: 0
```

¡LINQ es poderoso!

Hemos aprendido a consultar y transformar colecciones en memoria de forma elegante y declarativa.

Pero... ¿qué pasa cuando los datos NO están en memoria?

¿Podríamos usar una sintaxis parecida a LINQ para "hablar" con esos datos persistentes?

¡Sí! Y aquí es donde entra en juego la **persistencia de datos** y herramientas como **Entity Framework Core**.

¡A guardar y consultar datos de verdad!

Persistencia de datos

Vamos a usar SQLite para persistir datos

SQLite es una biblioteca que implementa un motor de base de datos SQL “en proceso”, autónomo, sin servidor, de configuración cero.

SQLite es de código abierto y, por lo tanto, es gratuito para su uso para cualquier propósito.





Crear una aplicación de consola llamada Escuela

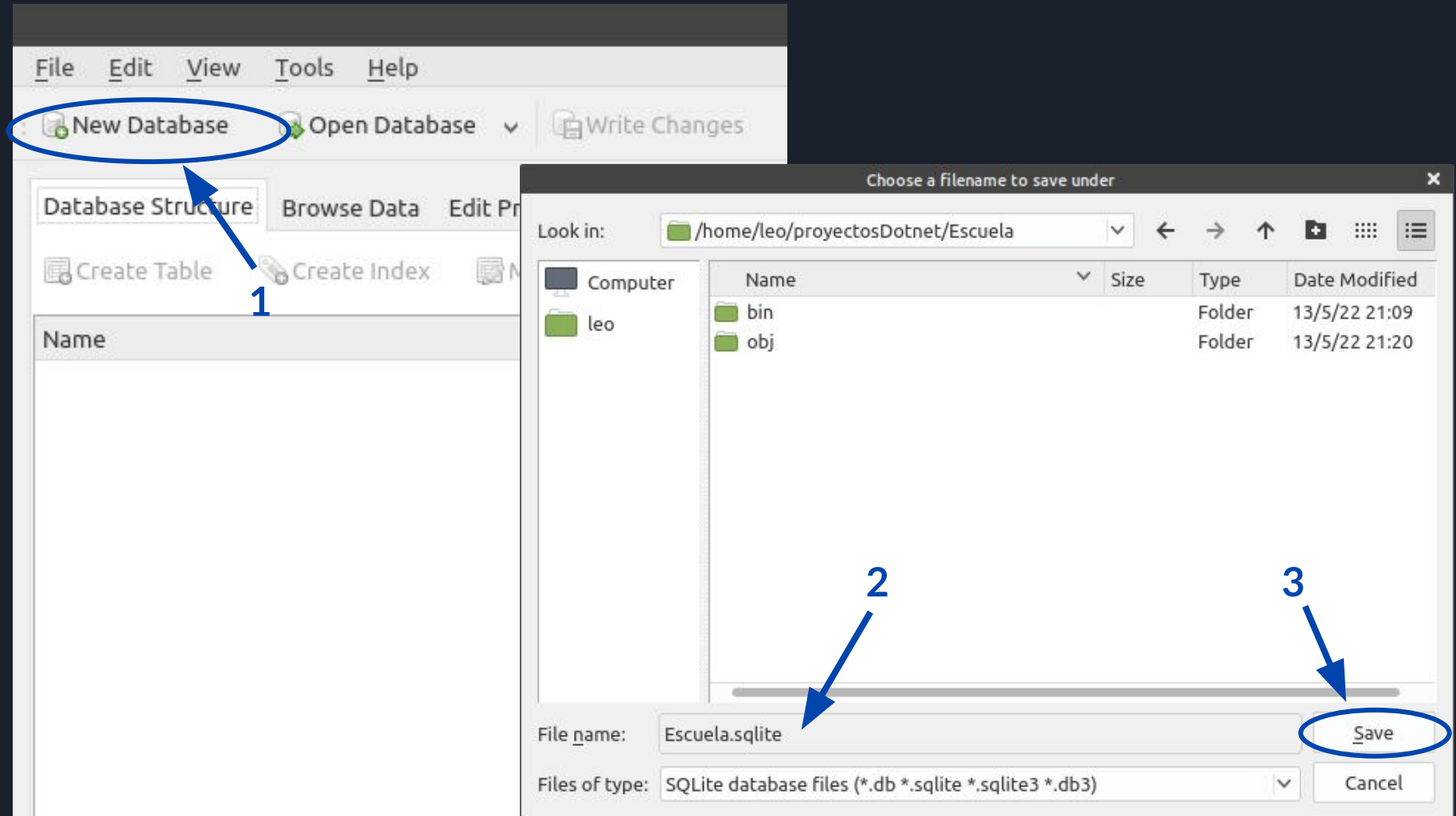


1. Abrir una terminal del sistema operativo
2. Cambiar a la carpeta `proyectosDotnet`
3. Crear la aplicación de consola `Escuela`
4. En la carpeta raíz del proyecto crear la base de datos `Escuela.sqlite` utilizando `DB Browser for SQLite`

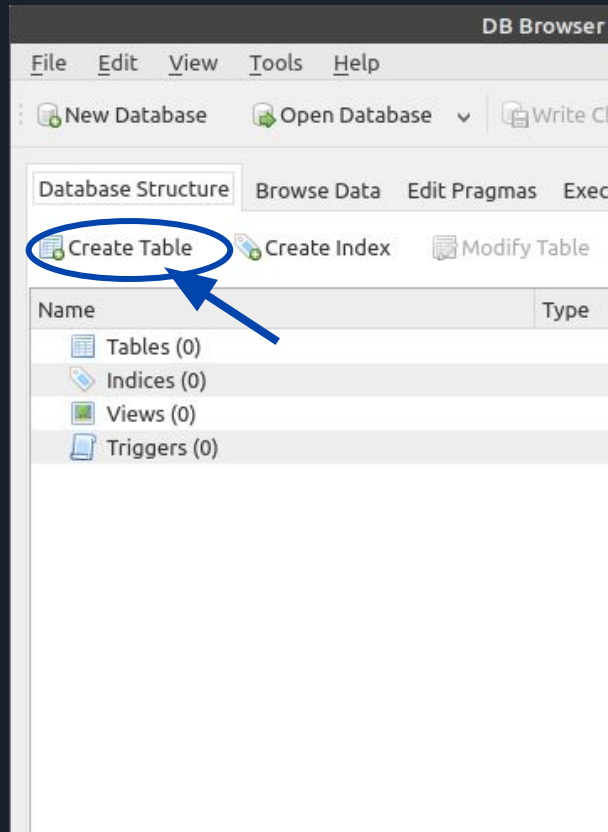
Descargar e instalar DB Browser for SQLite (<https://sqlitebrowser.org>)



Utilizando **DB Browser** crear la base **Escuela.sqlite** en la carpeta raíz del proyecto que acabamos de crear (donde se encuentra el archivo **Escuela.csproj**)



Crear la tabla **Alumnos** con los campos **Id** de tipo **INTEGER**, **Nombre** y **Email** de tipo **TEXT**



Marcamos el campo **Id** como clave de la tabla (PK). Además marcamos (AI) para que este campo sea establecido automáticamente por SQLite, de manera incremental, al momento de agregar una nueva fila a la tabla. Tanto **Nombre** como **Id** deben ser no nulos

Agregar algunos datos a la tabla Alumnos

The screenshot shows the SQLite GUI interface. The 'Browse Data' tab is selected, and the 'Alumnos' table is chosen. The table contains three records. A blue callout points to the 'Browse Data' tab, and another points to the 'New Record' button.

Pestaña Hoja de datos

Table: Alumnos

	Id	Nombre	Email
	Filter	Filter	Filter
1	1	Juan	juan@gmail.com
2	2	Ana	NULL
3	3	Laura	NULL

Presionando este botón se agrega un registro (fila) en blanco que podemos editar. El campo Id se completa solo

Crear la tabla **Exámenes** con los campos que se observan en esta diapositiva

Edit table definition

Table

Exámenes

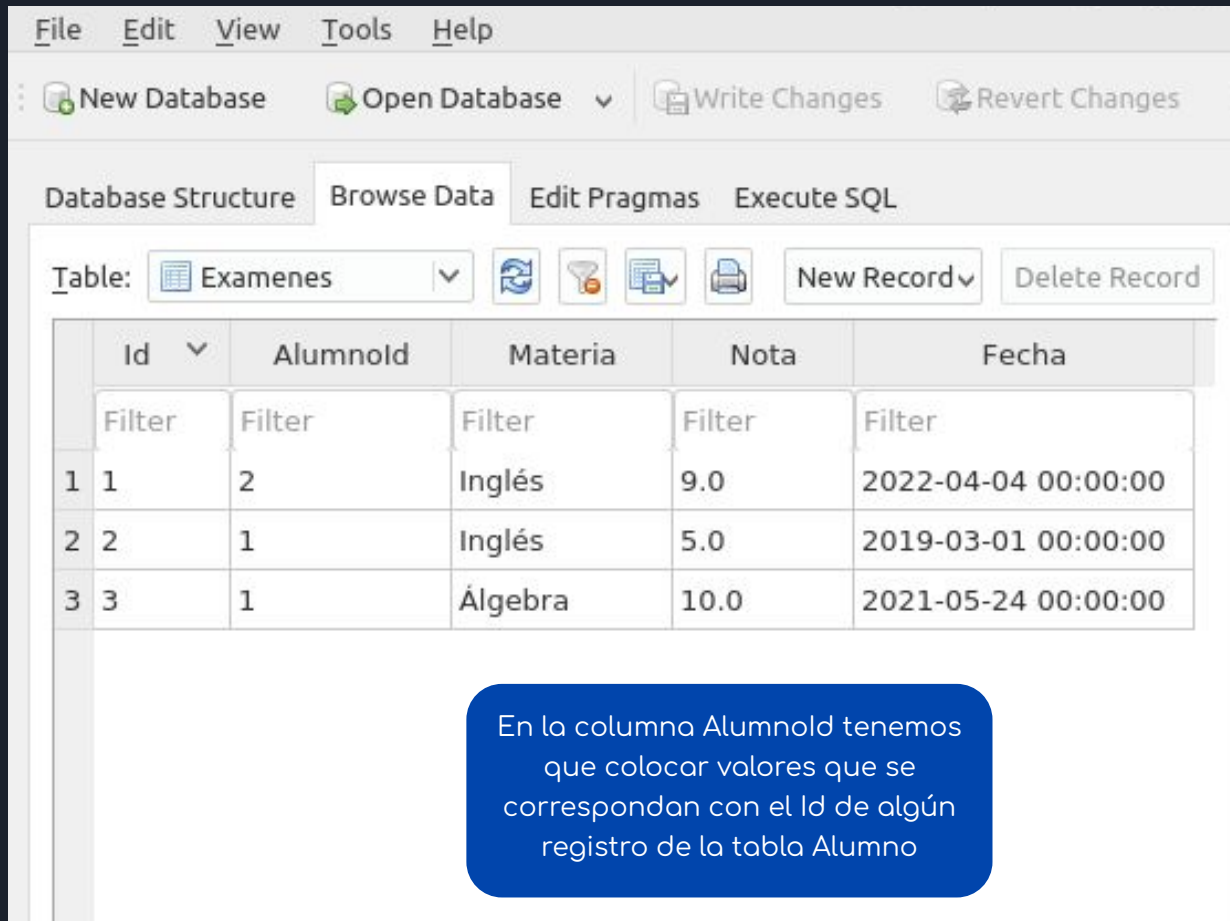
Advanced

Fields

Add field Remove field Move field up Move field down

Name	Type	NN	PK	AI	U	Default	Check	Foreign Key
Id	INTEGER	☒	☒	☒	☐			
Alumnoid	INTEGER	☒	☐	☐	☐			
Materia	TEXT	☒	☐	☐	☐			
Nota	REAL	☒	☐	☐	☐			
Fecha	TEXT	☒	☐	☐	☐			

Agregar algunos datos a la tabla Exámenes



The screenshot shows the SQLite Browser application with the 'Exámenes' table selected. The table has five columns: Id, Alumnold, Materia, Nota, and Fecha. The data is as follows:

	Id	Alumnold	Materia	Nota	Fecha
1	1	2	Inglés	9.0	2022-04-04 00:00:00
2	2	1	Inglés	5.0	2019-03-01 00:00:00
3	3	1	Álgebra	10.0	2021-05-24 00:00:00

A blue callout box contains the following text:

En la columna Alumnold tenemos que colocar valores que se correspondan con el Id de algún registro de la tabla Alumno

Entity Framework Core

Nuestra aplicación de consola accederá a los datos de **Escuela.sqlite** mediante **Entity Framework Core**.

EF Core es un **ORM** (object-relational mapper) que permite trabajar con una base de datos utilizando objetos .Net y ahorrando la escritura de mucho código de acceso a datos





Instalar el paquete Nuget Entity Framework Core para SQLite



En la terminal del sistema operativo (o en la que provee el **Visual Studio Code**) posicionarse en la carpeta del proyecto (donde se encuentra el archivo **Escuela.csproj**) y tipear el siguiente comando:

```
dotnet add package Microsoft.EntityFrameworkCore.Sqlite
```

Este es el nombre del paquete
que se requiere instalar

Copiar el código del archivo
10_RecursosParaLaTeoria

Entity Framework Core

Con **EF Core**, el acceso a datos se realiza mediante un **modelo**. Un modelo se compone de **clases de entidad** y un **objeto de contexto** que representa **una sesión con la base de datos**. Este objeto de contexto permite consultar y guardar datos.





Codificar la clases de entidad Alumno



```
namespace Escuela;

public class Alumno
{
    public int Id { get; private set; }
    public string Nombre { get; private set; } = "";
    public string? Email { get; private set; }

    public Alumno(string nombre, string? email = null)
    {
        if (string.IsNullOrEmpty(nombre))
        {
            throw new ArgumentException("El nombre no puede ser nulo ni estar vacío");
        }

        if (email != null && !EmailValido(email))
        {
            throw new ArgumentException("El formato del email no es válido.");
        }

        this.Nombre = nombre;
        this.Email = email;
        //El Id va a ser establecido por el sistema de persistencia
    }
    protected Alumno() { } // Constructor vacío (lo utilizará EntityFramework)

    private static bool EmailValido(string email)
    {
        // Una validación de email real puede ser más compleja
        return email.Contains('@') && email.Contains('.');
    }

    // Otros métodos de la entidad Alumno que implementan su comportamiento
    // permitiendo cambiar su estado y mantener su propia consistencia
    // es decir, las invariantes de la entidad (reglas que siempre deben ser verdaderas).
}
```

Copiar el código del archivo
10_RecursosParaLaTeoria

Persistencia de datos - SQLite - Entity Framework Core

```
namespace Escuela;

public class Alumno
{
    public int Id { get; private set; }
    public string Nombre { get; private set; } = "";
    public string? Email { get; private set; }

    public Alumno(string nombre, string? email = null)
    {
        if (string.IsNullOrEmpty(nombre))
        {
            throw new ArgumentException("El nombre no puede ser nulo ni estar vacío");
        }

        if (email != null && !EmailValido(email))
        {
            throw new ArgumentException("El formato del email no es válido.");
        }

        this.Nombre = nombre;
        this.Email = email;
        //El Id va a ser establecido por el sistema de persistencia
    }
    protected Alumno() { } // Constructor vacío (lo utilizará EntityFramework)

    private static bool EmailValido(string email)
    {
        // Una validación de email real puede ser más compleja
        return email.Contains('@') && email.Contains('.');
    }

    // Otros métodos de la entidad Alumno que implementan su comportamiento
    // permitiendo cambiar su estado y mantener su propia consistencia
    // es decir, las invariantes de la entidad (reglas que siempre deben ser verdaderas).
}
```

Observar que las propiedades se corresponden con la estructura de los datos guardados en la tabla **Alumnos** en la base de datos

```
namespace Escuela;
```

```
public class Alumno
```

```
{
```

```
    pu
```

```
    pu
```

```
    pu
```

```
    pu
```

```
{
```

¿Por qué este constructor vacío (**protected**)?

- **Plan de Respaldo del ORM:** Entity Framework Core intenta usar el constructor principal, pero en escenarios complejos (como al mapear *Value Objects*) exige uno sin parámetros para instanciar por reflexión, saltándose el modificador de acceso.
- **Filtración de Infraestructura (*Leaking Concern*):** Este es un claro ejemplo de cómo una necesidad técnica de la base de datos "se filtra" y ensucia nuestro Dominio.

"Es el precio que pagamos para que el ORM pueda
'hidratar' nuestras entidades automáticamente."

```
//El id va a ser establecido por el sistema de persistencia
```

```
}
```

```
protected Alumno() { } // Constructor vacío (lo utilizará EntityFramework)
```

```
private static bool EmailValido(string email)
```

```
{
```

```
    // Una validación de email real puede ser más compleja
```

```
    return email.Contains('@') && email.Contains('.');
```

```
}
```

```
// Otros métodos de la entidad Alumno que implementan su comportamiento
```

```
// permitiendo cambiar su estado y mantener su propia consistencia
```

```
// es decir, las invariantes de la entidad (reglas que siempre deben ser verdaderas).
```

```
}
```

Persistencia de datos - SQLite - Entity Framework Core

```
namespace Escuela;

public class Alumno
{
    public int Id { get; private set; }
    public string Nombre { get; private set; } = "";
    public string? Email { get; private set; }

    public Alumno(string nombre, string? email = null)
    {
        if (string.IsNullOrEmpty(nombre))
        {
            throw new ArgumentException("El nombre es obligatorio");
        }

        if (email != null)
        {
            throw new ArgumentException("El email no puede estar vacío");
        }

        this.Nombre = nombre;
        this.Email = email;
        //El Id se genera automáticamente por EF
    }
    protected virtual void OnModelCreating(ModelBuilder modelBuilder)
    {
        private static readonly string[] _validEmailDomains = { ".com", ".net", ".org", ".es" };

        // Una validacion de email real puede ser mas compleja
        return email.Contains('@') && email.Contains('.') && email.EndsWith(_validEmailDomains);
    }

    // Otros métodos de la entidad Alumno que implementan su comportamiento
    // permitiendo cambiar su estado y mantener su propia consistencia
    // es decir, las invariantes de la entidad (reglas que siempre deben ser verdaderas).
}
```

Usamos un parámetro opcional en el constructor. Un parámetro opcional permite establecer un valor por defecto para el caso de no especificarlo en la invocación

Para crear un alumno de nombre "Ana" sin email podemos hacer `new Alumno("Ana", null)` o bien simplemente `new Alumno("Ana");` gracias al parámetro opcional.

Persistencia de datos - SQLite - Entity Framework Core

```
namespace Escuela;

public class Alumno
{
    public int Id { get; private set; }
    public string Nombre { get; private set; } = "";
    public string? Email { get; private set; }

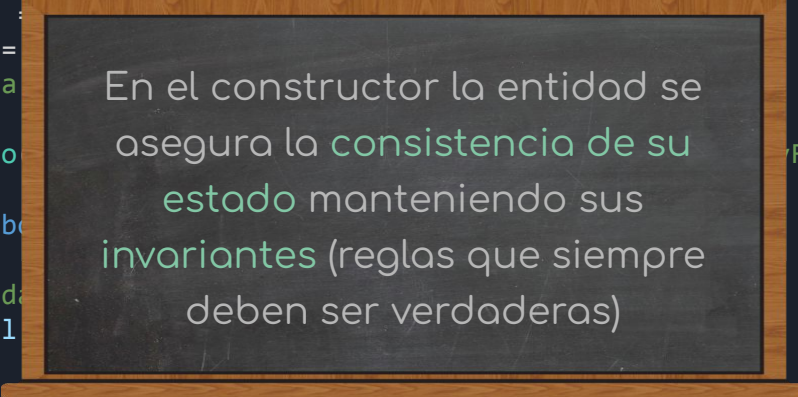
    public Alumno(string nombre, string? email = null)
    {
        if (string.IsNullOrEmpty(nombre))
        {
            throw new ArgumentException("El nombre no puede ser nulo ni estar vacío");
        }

        if (email != null && !EmailValido(email))
        {
            throw new ArgumentException("El formato del email no es válido.");
        }
    }
}
```

```
    this.Nombre = nombre;
    this.Email = email;
    //El Id va a ser asignado por el framework
}

protected Alumno()
{
    // Una validación de la entidad
    return email;
}

// Otros métodos de la entidad Alumno que implementan su comportamiento
// permitiendo cambiar su estado y mantener su propia consistencia
// es decir, las invariantes de la entidad (reglas que siempre deben ser verdaderas).
}
```



En el constructor la entidad se asegura la consistencia de su estado manteniendo sus invariantes (reglas que siempre deben ser verdaderas)

(Entity Framework)

Persistencia de datos - SQLite - Entity Framework Core

```
namespace Escuela;

public class Alumno
{
    public int Id { get; private set; }
    public string Nombre { get; private set; } = "";
    public string? Email { get; private set; }

    public Alumno(string nombre, string? email = null)
    {
        if (string.IsNullOrEmpty(nombre))
        {
            throw new ArgumentException("El nombre no puede estar vacío");
        }

        if (email != null && !EmailValido(email))
        {
            throw new ArgumentException("El formato del email no es válido.");
        }

        this.Nombre = nombre;
        this.Email = email;
        //El Id va a ser establecido por el sistema de persistencia
    }

    protected Alumno() { } // Constructor vacío (lo utilizará EntityFramework)

    private static bool EmailValido(string email)
    {
        // Una validación de email real puede ser más compleja
        return email.Contains('@') && email.Contains('.');
    }

    // Otros métodos de la entidad Alumno que implementan su comportamiento
    // permitiendo cambiar su estado y mantener su propia consistencia
    // es decir, las invariantes de la entidad (reglas que siempre deben ser verdaderas).
}
```

Establecemos sólo las propiedades **Nombre** y **Email**.
El **Id** será generado por el sistema de persistencia

```
namespace Escuela;

public class Alumno
{
    public int Id { get; private set; }
    public string Nombre { get; private set; } = "";
    public string? Email { get; private set; }
```

Identidad en DDD y "Leaking Concerns"

- **El problema de la identidad delegada:** Desde una perspectiva purista de **DDD**, que la identidad de una entidad dependa del momento de su persistencia es un **leaking concern: una filtración de responsabilidades**. Un interés tecnológico de la infraestructura termina condicionando al dominio.
- **Entidades incompletas:** Un objeto obtiene su **Id** después de guardarse en la base de datos, estará incompleto hasta ese momento.
- **El ideal en DDD:** El dominio debe estar **aislado** de **bases de datos** y **frameworks**. La entidad debería generar su propia identidad en el momento de su creación, comúnmente utilizando el tipo **Guid**.

Establecer sólo las
Nombre y Email.
por el sistema
encia

```
// es decir, las invariantes de la entidad (reglas que siempre deben ser verdaderas).
```

```
}
```



```
namespace Escuela;

public class Alumno
{
    public int Id { get; private set; }
    public string Nombre { get; private set; } = "";
    public string? Email { get; private set; }
```

Trade-Off: Pureza del Dominio vs. Rendimiento

- **El costo del Guid:** Utilizar **Guid** en lugar de **int** puede afectar el rendimiento de los índices en bases de datos muy grandes o con altísima concurrencia.
- **La decisión pragmática:** Si el rendimiento de la base de datos es el factor más crítico del sistema, aceptar la "impureza" de un **Id autoincremental establecido post-persistencia** es un compromiso válido y muy común en la industria.
- **La decisión purista:** En proyectos donde se prioriza un **modelo de dominio rico, testeable** y libre de dependencias, la balanza se inclina fuertemente hacia los **Guids** generados en memoria.

Establecemos sólo las
Nombre y Email.
por el sistema
encia

```
// es decir, las invariantes de la entidad (reglas que siempre deben ser verdaderas).
```

```
}
```

```
namespace Escuela;

public class Alumno
{
    public int Id { get; private set; }
    public string Nombre { get; private set; } = "";
    public string? Email { get; private set; }
```

Enfoque de esta Clase vs. Trabajo Final

- **En el ejemplo de hoy:** Por cuestiones didácticas, veremos cómo utilizar **Entity Framework** con **Ids** de tipo **int** generados por la persistencia de manera autoincremental. El objetivo es mostrar cómo se implementa esta alternativa pragmática ampliamente utilizada en la industria.
- **IMPORTANTE - Trabajo Final:** Para el proyecto de este curso, la regla será mantener alta la pureza del modelo. Seguiremos utilizando el enfoque **DDD** donde las propias entidades generen sus **Ids** utilizando **Guid**, asegurando que el dominio no dependa de la infraestructura.

Estableceremos sólo las
Nombre y Email.
por el sistema
encia

```
// es decir, las invariantes de la entidad (reglas que siempre deben ser verdaderas).
```

```
}
```

```
namespace Escuela;

public class Examen
{
    public int Id { get; private set; }
    public int AlumnoId { get; private set; }
    public string Materia { get; private set; } = "";
    public double Nota { get; private set; }
    public DateTime Fecha { get; private set; }
    public Examen(int alumnoId, string materia, double nota, DateTime fecha)
    {
        // completar aquí las validaciones que aseguren la consistencia de la entidad
        AlumnoId = alumnoId;
        Materia = materia;
        Nota = nota;
        Fecha = fecha;
    }

    protected Examen() { } // Constructor para EF Core (leaking concern)

    public void CambiarNota(double nota)
    {
        if (nota < 0 || nota > 10)
        {
            throw new ArgumentOutOfRangeException("Valor para la nota no permitido");
        }
        Nota = nota;
    }

    // Otros métodos que implementan el comportamiento de la entidad
    // manteniendo sus invariantes
}
```



Codificar la clases de entidad Examen



Copiar el código del archivo
10_RecursosParaLaTeoria



Codificar la clase EscuelaContext



```
using Microsoft.EntityFrameworkCore;

namespace Escuela;

public class EscuelaContext : DbContext
{

    public DbSet<Alumno> Alumnos { get; set; }
    public DbSet<Examen> Exámenes { get; set; }

    protected override void OnConfiguring(DbContextOptionsBuilder
        optionsBuilder)
    {
        optionsBuilder.UseSqlite("data source=Escuela.sqlite");
    }
}
```

Esta clase debe heredar de `DbContext` e invalidar el método `OnConfiguring` que se utiliza para identificar la fuente de datos

Copiar el código del archivo
10_RecursosParaLaTeoria

El nombre de estas propiedades coincide con el nombre de las tablas en la base de datos



Codificar la clase EscuelaContext



```
using Microsoft.EntityFrameworkCore;
```

```
namespace Escuela;
```

```
public class EscuelaContext : DbContext
{
    #nullable disable
    public DbSet<Alumno> Alumnos { get; set; }
    public DbSet<Examen> Exámenes { get; set; }
    #nullable restore
```

Las propiedades `Alumnos` y `Exámenes` serán inicializadas por `Entity Framework`. Si el compilador arroja **warnings** (versiones anteriores) podemos deshabilitar el contexto nullable en esta sección

```
protected override void OnConfiguring(DbContextOptionsBuilder
    optionsBuilder)
{
    optionsBuilder.UseSqlite("data source=Escuela.sqlite");
}
}
```

Invalidando `OnConfiguring` podemos establecer el motor de la base de datos y el archivo



Codificar Program.cs de la siguiente manera



```
using Escuela;

using (var context = new EscuelaContext())
{
    Console.WriteLine("-- Tabla Alumnos --");
    foreach (var a in context.Alumnos)
    {
        Console.WriteLine($"{a.Id} {a.Nombre}");
    }

    Console.WriteLine("-- Tabla Exámenes --");
    foreach (var ex in context.Examenes)
    {
        Console.WriteLine($"{ex.Id} {ex.Materia} {ex.Nota}");
    }
}
```

Copiar el código del archivo
10_RecursosParaLaTeoria

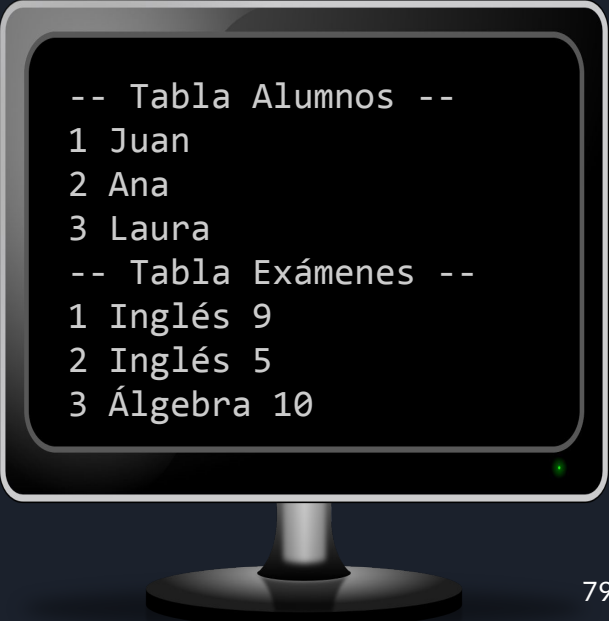

```
using Escuela;

using (var context = new EscuelaContext())
{
    Console.WriteLine("-- Tabla Alumnos --");
    foreach (var a in context.Alumnos)
    {
        Console.WriteLine($"{a.Id} {a.Nombre}");
    }

    Console.WriteLine("-- Tabla Exámenes --");
    foreach (var ex in context.Examenes)
    {
        Console.WriteLine($"{ex.Id} {ex.Materia} {ex.Nota}");
    }
}
```

Un objeto `DbContext` está diseñado para usarse durante una única unidad de trabajo, por lo que la duración de una instancia de un `DbContext` suele ser muy breve.

Es importante invocar al método `Dispose()` al terminar de usar el `DbContext` (en este caso lo realiza la instrucción `using`)



```
-- Tabla Alumnos --
1 Juan
2 Ana
3 Laura
-- Tabla Exámenes --
1 Inglés 9
2 Inglés 5
3 Álgebra 10
```

Entity Framework Core - Code First

Si empezamos un proyecto nuevo, para el cual no existe una base de datos creada, podemos crearla fácilmente a partir del código C# que ya tenemos codificado. A esta estrategia se la conoce con el nombre de “Code First”





Codificar la siguiente clase que usaremos para crear e inicializar la base de datos desde cero



```
namespace Escuela;

public class EscuelaSqlite
{
    public static void Inicializar()
    {
        using var context = new EscuelaContext();
        if (context.Database.EnsureCreated())
        {
            Console.WriteLine("Se creó base de datos");
        }
    }
}
```

Podemos utilizar una declaración using en lugar de la instrucción using

Si la base de datos no existe, se crea según el modelo definido en `EscuelaContext` y devuelve `true`

Copiar el código del archivo
10_RecursosParaLaTeoria



Borrar la base de datos **Escuela.sqlite** del proyecto, agregar la línea indicada en Program.cs y ejecutar



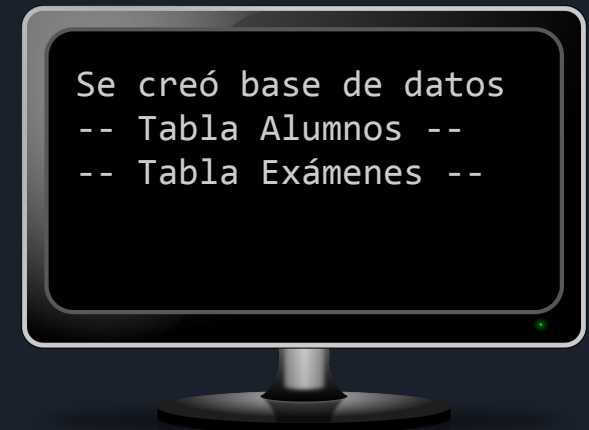
```
using Escuela;
```

```
EscuelaSqlite.Inicializar();
```



```
using (var context = new EscuelaContext())
{
    Console.WriteLine("-- Tabla Alumnos --");
    foreach (var a in context.Alumnos)
    {
        Console.WriteLine($"{a.Id} {a.Nombre}");
    }

    Console.WriteLine("-- Tabla Exámenes --");
    foreach (var ex in context.Examenes)
    {
        Console.WriteLine($"{ex.Id} {ex.Materia} {ex.Nota}");
    }
}
```



Persistencia de datos - SQLite - Entity Framework Core

Table:

Advanced

Fields

Add field Remove field Move field up Move field down

Name	Type	NN	PK	AI	U
Id	INTEGER	☒	☒	☒	☐
Nombre	TEXT	☒	☐	☐	☐
Email	TEXT	☐	☐	☐	☐

```
1 CREATE TABLE "Alumnos" (  
2     "Id" INTEGER NOT NULL PRIMARY KEY AUTOINCREMENT,  
3     "Nombre" TEXT NOT NULL,  
4     "Email" TEXT  
5 );
```

Cancel

Table:

Advanced

Fields

Add field Remove field Move field up Move field down

Name	Type	NN	PK	AI	U	Default	Check
Id	INTEGER	☒	☒	☒	☐		
AlumnoId	INTEGER	☒	☐	☐	☐		
Materia	TEXT	☒	☐	☐	☐		
Nota	REAL	☒	☐	☐	☐		
Fecha	TEXT	☒	☐	☐	☐		

```
1 CREATE TABLE "Exámenes" (  
2     "Id" INTEGER NOT NULL PRIMARY KEY AUTOINCREMENT,  
3     "AlumnoId" INTEGER NOT NULL,  
4     "Materia" TEXT NOT NULL,  
5     "Nota" REAL NOT NULL,  
6     "Fecha" TEXT NOT NULL  
7 );
```

Cancel OK

Las tablas **Alumnos** y **Exámenes** se crearon con estas configuraciones a partir de nuestro modelo utilizando ciertas convenciones por defecto

Table:

Advanced

Fields

Add field Remove field Move field up Move field down

Name	Type	NN	PK	AI	U
Id	INTEGER	☒	☒	☒	☐
Nombre	TEXT	☒	☐	☐	☐
Email	TEXT	☐	☐	☐	☐

```
1 CREATE TABLE "Alumnos" (  
2     "Id" INTEGER NOT NULL PRIMARY KEY AUTOINCREMENT,  
3     "Nombre" TEXT NOT NULL,  
4     "Email" TEXT  
5 );
```

Table:

Advanced

Fields

Add field Remove field Move field up Move field down

Name	Type	NN	PK	AI	U	Default	Check
Id	INTEGER	☒	☒	☒	☐		
AlumnoId	INTEGER	☒	☐	☐	☐		
Materia	TEXT	☒	☐	☐	☐		
Nota	REAL	☒	☐	☐	☐		
Fecha	TEXT	☒	☐	☐	☐		

```
1 CREATE TABLE "Exámenes" (  
2     "Id" INTEGER NOT NULL PRIMARY KEY AUTOINCREMENT,  
3     "AlumnoId" INTEGER NOT NULL,  
4     "Materia" TEXT NOT NULL,  
5     "Nota" REAL NOT NULL,  
6     "Fecha" TEXT NOT NULL  
7 );
```

Cancel OK

Convenciones por defecto utilizadas:

- El **nombre de las tablas** se determinó a partir del nombre de las propiedades `DbSet<>` de `EscuelaContext`
- La propiedad `Id` de las entidades se establecen como **claves** de la tablas
- Si la propiedad `Id` de una entidad es **entera**, se establece como **clave autoincremental**
- Sqlite soporta pocos tipos de datos, se utiliza un mapeo adecuado, por ejemplo las propiedades `DateTime` de .NET se establecen como campos **TEXT** en las tablas **Sqlite**

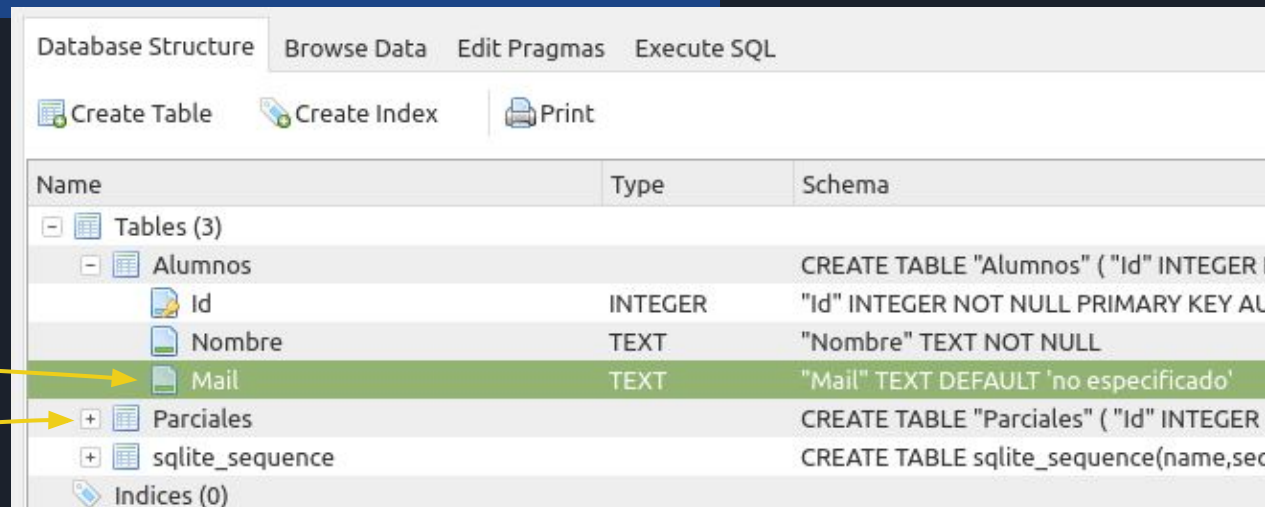
Persistencia de datos - SQLite - Entity Framework Core

```
public class EscuelaContext : DbContext
{
    #nullable disable
    public DbSet<Alumno> Alumnos { get; set; }
    public DbSet<Examen> Examenes { get; set; }
    #nullable restore
```

Se podría invalidar **OnModelCreating** para especificar distintos aspectos sobre cómo se realiza el mapeo entre el modelo y la base de datos utilizando Fluent API (es sólo ilustrativo, no codificarlo para seguir estas diapositivas)

```
protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
{
    optionsBuilder.UseSqlite("data source=Escuela.sqlite");
}
```

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Examen>().ToTable("Parciales");
    modelBuilder.Entity<Alumno>()
        .Property(a => a.Email)
        .HasColumnName("Mail").HasDefaultValue("no especificado");
}
```



Name	Type	Schema
Tables (3)		
Alumnos		
Id	INTEGER	"Id" INTEGER NOT NULL PRIMARY KEY AU
Nombre	TEXT	"Nombre" TEXT NOT NULL
Mail	TEXT	"Mail" TEXT DEFAULT 'no especificado'
Parciales		CREATE TABLE "Parciales" ("Id" INTEGER
sqlite_sequence		CREATE TABLE sqlite_sequence(name,seq
Indices (0)		

Persistencia de datos - SQLite - Entity Framework Core

The screenshot shows the Visual Studio Code interface with the 'Escuela.sqlite - Escuela - Visual Studio Code' window. The 'EXTENSIONS: MARKETPLACE' sidebar is open, displaying a list of extensions. The 'SQLite Viewer' extension by Florian Klampfer is highlighted with a yellow box. A yellow arrow points from this extension to a blue callout box on the right. The callout box contains the text: 'Para visualizar el contenido de la base de datos en Visual Studio Code instalar alguna extensión para SQLite (por ejemplo SQLite Viewer o SQLite3 Editor)'. The background shows the 'Escuela.sqlite' database with tables 'Alumnos', 'sqlite_sequence', and 'Exámenes'.

Escuela.sqlite - Escuela - Visual Studio Code

File Edit Selection View Go Run Terminal Help

EXTENSIONS: MARKETPLACE

sqlite

SQLite Viewer 3ms
SQLite Viewer for VSCode
Florian Klampfer

SQLite & MySQL Snippets 13K 5
A snippet for MySQL and SQLite...
Rohit Chouhan Install

SQLTools 1.7M 3.5
Database management done right...
Matheus Teixeira Install

Flutter ORM M8 Generator 15K
Vs Code generator for orm-m8 f...
Mircea-Tiberiu MATEI Install

dBizzy 2K 4
ER Diagram Visualizer
dBizzy Install

GrayCat SQL Formatter 8K 3
A VSCode formatter extension f...
Adam Rybak Install

MySQL 499K 4.5
Database manager for MySQL/...
cweijan Install

Database Client 62K 5
Database manager for MySQL/...

Search tables.. Reset Filters Records: 0 Search 0 records...

Tables (3)

- Alumnos
- sqlite_sequence
- Exámenes

Id Nombre Email

Search column... Search column... Search column...

Para visualizar el contenido de la base de datos en Visual Studio Code instalar alguna extensión para SQLite (por ejemplo SQLite Viewer o SQLite3 Editor)

0 0 .NET Core Launch (console) (Escuela) Escuela

Page 1 / 1 Try SQLite Viewer Web

Persistencia de datos - SQLite - Entity Framework Core

Escuela.sqlite - Escuela - Visual Studio Code

File Edit Selection View Go Run Terminal Help

EXPLORER

- ESCUELA
 - .vscode
 - bin
 - obj
 - Alumno.cs
 - Escuela.csproj
 - Escuela.sqlite
 - EscuelaContext.cs
 - EscuelaIni.cs
 - Examen.cs
 - Program.cs

Escuela.sqlite

Search tables... Reset Filters Records: 0 Search 0 records...

Tables (3)

- Alumnos
 - Id
 - Nombre
 - Email
- sqlite_sequence
- Examenes

Visualizando la base de datos Escuela.sqlite

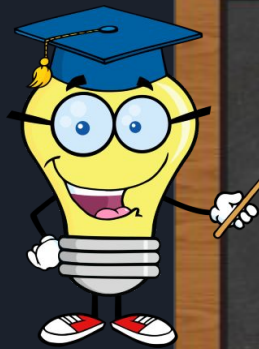
En Escuela.sqlite podemos ver las tablas Alumnos y Examenes creadas de acuerdo al modelo definido

Try SQLite Viewer Web

Page 1 / 1

.NET Core Launch (console) (Escuela) Escuela

Entity Framework Core - Code First



Existe un mecanismo más sofisticado para implementar “Code First” que se llama migraciones.

Con las **migraciones** es posible realizar modificaciones incrementales en la base de datos e incluso volver para atrás removiendo la última migración

Para utilizarlas es necesario instalar las herramientas de EF necesarias



Modificar la clase `EscuelaSqlite` para que al crear la base agregue algunos alumnos y exámenes



```
namespace Escuela;

public class EscuelaSqlite
{
    public static void Inicializar()
    {
        using var context = new EscuelaContext();
        if (context.Database.EnsureCreated())
        {
            Console.WriteLine("Se creó base de datos");

            context.Alumnos.Add(new Alumno("Juan", "juan@gmail.com"));
            context.Alumnos.Add(new Alumno("Ana"));
            context.Alumnos.Add(new Alumno("Laura"));

            context.Examenes.Add(new Examen(2, "Inglés", 9, new DateTime(2022, 4, 4)));
            context.Examenes.Add(new Examen(1, "Inglés", 5, new DateTime(2019, 3, 1)));
            context.Examenes.Add(new Examen(1, "Álgebra", 10, new DateTime(2021, 5, 24)));

            context.SaveChanges();
        }
    }
}
```

Copiar el código del archivo
10_RecursosParaLaTeoria



Modificar la clase `EscuelaSqlite` para que al crear la base agregue algunos alumnos y exámenes



```
namespace Escuela;

public class EscuelaSqlite
{
    public static void Inicializar()
    {
        using var context = new EscuelaContext();
        if (context.Database.EnsureCreated())
        {
            Console.WriteLine("Se creó base de datos");

            context.Add(new Alumno("Juan", "juan@gmail.com"));
            context.Add(new Alumno("Ana"));
            context.Add(new Alumno("Laura"));

            context.Add(new Examen(2, "Inglés", 9, new DateTime(2022, 4, 4)));
            context.Add(new Examen(1, "Inglés", 5, new DateTime(2019, 3, 1)));
            context.Add(new Examen(1, "Álgebra", 10, new DateTime(2021, 5, 24)));

            context.SaveChanges();
        }
    }
}
```

Esta instrucción actualiza en la base de datos todos los cambios realizados (en este caso el alta de los alumnos y exámenes realizados)

Observar que podemos no especificar la propiedad `DbSet<>` donde agregar un `Alumno` o un `Examen`, porque existe una única propiedad `DbSet<>` por cada entidad.



Modificar la clase **EscuelaSqlite** para que al crear la base agregue algunos alumnos y exámenes



```
Se creó base de datos
-- Tabla Alumnos --
1 Juan
2 Ana
3 Laura
-- Tabla Exámenes --
1 Inglés 9
2 Inglés 5
3 Álgebra 10
```

Persistencia de datos - SQLite - Entity Framework Core

Database Structure Browse Data Edit Pragmas Execute SQL

Table: Alumnos

New Record Delete Record

	Id	Nombre	Email
	Filter	Filter	Filter
1	1	Juan	juan@gmail..
2	2	Ana	NULL
3	3	Laura	NULL

1 - 3 of 3 Go to:

Database Structure Browse Data Edit Pragmas Execute SQL

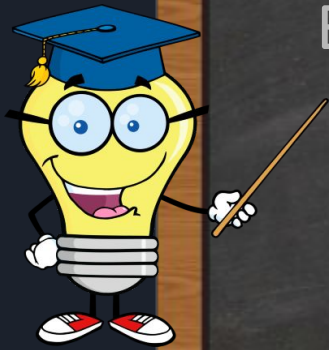
Table: Exámenes

New Record Delete Record

	Id	Alumnoid	Materia	Nota	Fecha
	Filter	Filter	Filter	Filter	Filter
1	1	2	Inglés	9.0	2022-04-04 ...
2	2	1	Inglés	5.0	2019-03-01 ...
3	3	1	Álgebra	10.0	2021-05-24 ...

1 - 3 of 3 Go to: 1

Entity Framework Core - Code First



Está claro que existe una relación “uno a muchos” entre Alumnos y Exámenes

Alumnos

Nombre	Id
Juan	1
Ana	2
Laura	3

Exámenes

Alumnoid	Materia	Nota	Id
2	Inglés	9	1
1	Inglés	5	2
1	Álgebra	10	3

Por cada alumno podemos tener 0, 1 o más exámenes



Modificar Program.cs y ejecutar

```
using Escuela;
```

```
EscuelaSqlite.Inicializar(); //solo tiene efecto si  
                             // la base de datos no existe
```

```
using var context = new EscuelaContext();  
var query = context.Alumnos.Join(context.Examenes,  
    a => a.Id,  
    e => e.AlumnoId,  
    (a, e) => new  
    {  
        Alumno = a.Nombre,  
        Materia = e.Materia,  
        Nota = e.Nota  
    });
```

```
foreach (var obj in query)  
{  
    Console.WriteLine(obj);  
}
```

Consultando
datos



Utilizamos LINQ
para realizar
consultas

Copiar el código del archivo
10_RecursosParaLaTeoria

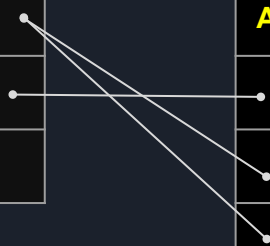
```
...  
  
var query = context.Alumnos.Join(context.Examenes,  
    a => a.Id,  
    e => e.AlumnoId,  
    (a, e) => new  
    {  
        Alumno = a.Nombre,  
        Materia = e.Materia,  
        Nota = e.Nota  
    });  
foreach (var obj in query)  
{  
    Console.WriteLine(obj);  
}
```

Alumnos

Nombre	Id
Juan	1
Ana	2
Laura	3

Examenes

Alumnoid	Materia	Nota	Id
2	Inglés	9	1
1	Inglés	5	2
1	Álgebra	10	3



```
{ Alumno = Ana, Materia = Inglés, Nota = 9 }  
{ Alumno = Juan, Materia = Inglés, Nota = 5 }  
{ Alumno = Juan, Materia = Álgebra, Nota = 10 }
```

```
...  
  
var query = context.Alumnos.Join(context.Examenes,  
    a => a.Id,  
    e => e.AlumnoId,  
    (a, e) => new  
    {  
        Alumno = a.Nombre,  
        Materia = e.Materia,  
        Nota = e.Nota  
    });  
foreach (var obj in query)  
{  
    Console.WriteLine(obj);  
}
```

Alumnos

Nombre	Id
Juan	1
Ana	2
Laura	3

Exámenes

AlumnoId	Materia	Nota	Id
2	Inglés	9	1
1	Inglés	5	2
1	Álgebra	10	3

Para los que conocen SQL

Esta es la instrucción SQL que generó Entity Framework para recuperar los datos desde la base

```
SELECT "a"."Nombre" AS "Alumno", "e"."Materia", "e"."Nota"  
FROM "Alumnos" AS "a"  
INNER JOIN "Exámenes" AS "e" ON "a"."Id" = "e"."AlumnoId"
```

La consulta a la base de datos se realiza recién al ejecutar el `foreach`



Modificar Program.cs y ejecutar



```
using Escuela;

EscuelaSqlite.Inicializar(); //solo tiene efecto si
                             // la base de datos no existe

using var context = new EscuelaContext();

//Agregamos un nuevo alumno
var alumno = new Alumno("Pablo"); // el Id será establecido por SQLite
context.Add(alumno); // se agregará realmente con el db.SaveChanges()

context.SaveChanges(); // actualiza la base de datos.
                       // SQLite establece el valor de alumno.Id

// Agregamos un examen para el nuevo alumno
var examen = new Examen(alumno.Id, "Historia", 9.5, DateTime.Today);
context.Add(examen);

context.SaveChanges();
```

Agregando
registros

Copiar el código del archivo
10_RecursosParaLaTeoria

Estado de las tablas luego de ejecutar el código

```
using Escuela;
```

```
EscuelaSqlite.Inicializar(); //solo tiene efecto si
                             // la base de datos no existe
```

```
using var context
```

```
//Agregamos un nue
var alumno = new A
context.Add(alumno
```

```
context.SaveChanges
```

```
// Agregamos un exa
var examen = new Examen(alumno.Id, "Historia", 9.5, DateTime.Today);
context.Add(examen);
```

```
context.SaveChanges();
```

Así queda la base de datos

Alumnos		Exámenes			
Nombre	Id	Alumnoid	Materia	Nota	Id
Juan	1				
Ana	2	2	Inglés	9	1
Laura	3	1	Inglés	5	2
Pablo	4	1	Álgebra	10	3
		4	Historia	9.5	4



Modificar Program.cs y ejecutar



```
using Escuela;
```

```
EscuelaSqlite.Inicializar(); //solo tiene efecto si  
                             // la base de datos no existe
```

```
using var context = new EscuelaContext();
```

```
//borramos de la tabla Alumnos el registro con Id=3
```

```
var alumnoBorrar = context.Alumnos.Where(a => a.Id == 3).SingleOrDefault();
```

```
if (alumnoBorrar != null)
```

```
{
```

```
    context.Remove(alumnoBorrar); //se borra realmente con el context.SaveChanges()
```

```
}
```

```
//La nota en Inglés del alumno id=1 es un 5. La cambiamos a 7.5
```

```
var examenModificar = context.Examenes.Where(
```

```
    e => e.AlumnoId == 1 && e.Materia == "Inglés").SingleOrDefault();
```

```
if (examenModificar != null)
```

```
{
```

```
    examenModificar.CambiarNota(7.5); //se modifica el registro en memoria
```

```
}
```

```
context.SaveChanges(); //actualiza la base de datos.
```

Borrando y
actualizando
registros

Copiar el código del archivo
10_RecursosParaLaTeoria

Modificar Program.cs y ejecutar

```
using Escuela;
```

```
EscuelaSqlite.Inicializar(); //solo tiene efecto si
                             // la base de datos no existe
```

```
using var context = new EscuelaContext();
```

```
//borramos de la tabla Alumnos el registro con Id=3
```

```
var alumnoBorrar = context.Alumnos.Where(a => a.Id == 3).SingleOrDefault();
```

```
if (alumnoBorrar != null)
```

```
{
```

```
context
```

```
}
```

```
//La nota
```

```
var examen
```

```
if (examen
```

```
{
```

```
examen
```

```
}
```

```
context.SaveChanges(); //actualiza la base de datos.
```

Devuelve el único elemento de la secuencia. Si no hay ninguno devuelve el valor por defecto (null en este caso). Si no es único lanza una excepción

Así queda la base de datos

Alumnos	
Nombre	Id
Juan	1
Ana	2
Pablo	4

Exámenes

Alumnoid	Materia	Nota	Id
2	Inglés	9	1
1	Inglés	7.5	2
1	Álgebra	10	3
4	Historia	9.5	4

```
anges()
```

```
SingleOrDefault();
```

Value Objects y Entity Framework Core

Codificar el siguiente Value Object:

```
public record class DireccionEmail
{
    // Inicialización en línea para satisfacer al compilador
    public string Cuenta { get; private init; } = "";
    public string Dominio { get; private init; } = "";

    public DireccionEmail(string cuenta, string dominio)
    {
        if (string.IsNullOrEmpty(cuenta) || string.IsNullOrEmpty(dominio))
        {
            throw new ArgumentException("La cuenta y el dominio son obligatorios");
        }
        Cuenta = cuenta;
        Dominio = dominio;
    }

    // Constructor vacío para EF Core (Leaking Concern)
    protected DireccionEmail() { }

    public override string ToString()
    {
        return $"{Cuenta}@{Dominio}";
    }
}
```



Value Objects y Entity Framework Core

Modificar la clase Alumno:

```
public class Alumno
{
    // ... otras propiedades ...

    public DireccionEmail? Email { get; private set; }

    public Alumno(string nombre, DireccionEmail? email = null)
    {
        if (string.IsNullOrWhiteSpace(nombre))
        {
            throw new ArgumentException("El nombre no puede ser nulo ni estar vacío");
        }

        this.Nombre = nombre;
        this.Email = email;
        //El Id va a ser establecido por el sistema de persistencia
    }
    // ...
}
```



Value Objects y Entity Framework Core

Modificar la clase EscuelaContext (invalidando el método OnModelCreating):

```
public class EscuelaContext : DbContext
{

    public DbSet<Alumno> Alumnos { get; set; }
    public DbSet<Examen> Examenes { get; set; }

    protected override void OnConfiguring(DbContextOptionsBuilder
        optionsBuilder)
    {
        optionsBuilder.UseSqlite("data source = Escuela.sqlite");
    }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);
        // Le indicamos a EF Core que Email es un "Tipo Complejo" (Value Object)
        modelBuilder.Entity<Alumno>().ComplexProperty(a => a.Email);
    }
}
```



Value Objects y Entity Framework Core

Modificar el método Inicializar de EscuelaSqlite

```
public class EscuelaSqlite
{
    public static void Inicializar()
    {
        using var context = new EscuelaContext();
        if (context.Database.EnsureCreated())
        {
            Console.WriteLine("Se creó base de datos");

            context.Alumnos.Add(new Alumno("Juan", new DireccionEmail("juan", "gmail.com")));
            context.Alumnos.Add(new Alumno("Ana"));
            context.Alumnos.Add(new Alumno("Laura"));

            context.Examenes.Add(new Examen(2, "Inglés", 9, new DateTime(2022, 4, 4)));
            context.Examenes.Add(new Examen(1, "Inglés", 5, new DateTime(2019, 3, 1)));
            context.Examenes.Add(new Examen(1, "Álgebra", 10, new DateTime(2021, 5, 24)));

            context.SaveChanges();
        }
    }
}
```



Value Objects y Entity Framework Core

Modificar el método Inicializar de EscuelaSqlite

```
public class EscuelaSqlite
{
    public static void Inicializar()
    {
```

```
        using va
        if (cont
        {
```

```
            Cons
```

```
            cont
```

```
            cont
```

```
            cont
```

```
            cont
```

```
            cont
```

```
            cont
```

```
        context.SaveChanges();
```

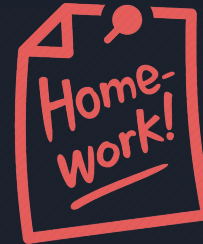
```
    }
```

```
}
```

```
}
```

Borrar la base de datos
Escuela.sqlite y ejecutar

Con una extensión adecuada en Visual Studio Code o con DB Browser for SQLite, observar las columnas de la tabla Alumnos generada



```
        "juan", "gmail.com"))));
```

```
        me(2022, 4, 4)));
```

```
        me(2019, 3, 1)));
```

```
        Time(2021, 5, 24)));
```

Fin de
Teoría 10

Práctica sobre la teoría 10

Práctica sobre la teoría 10

1) Utilizando el método **Range** de la clase **System.Linq.Enumerable** y los métodos de **LINQ** que sean necesarios, obtener:

- a) Lista con todos los múltiplos de 5 entre 100 y 200
- b) Lista con todos los números primos menores que 100
- c) Lista con las potencias de 2, desde 2^0 a 2^{10}
- d) La suma y el promedio de los valores de la lista anterior
- e) Lista de todos los n^2 que terminan con el dígito 6, para n entre 1 y 20
- f) Lista con los nombres de los días de la semana en inglés que contengan una letra 'u' (tip: utilizar el enumerativo **DayOfWeek**)

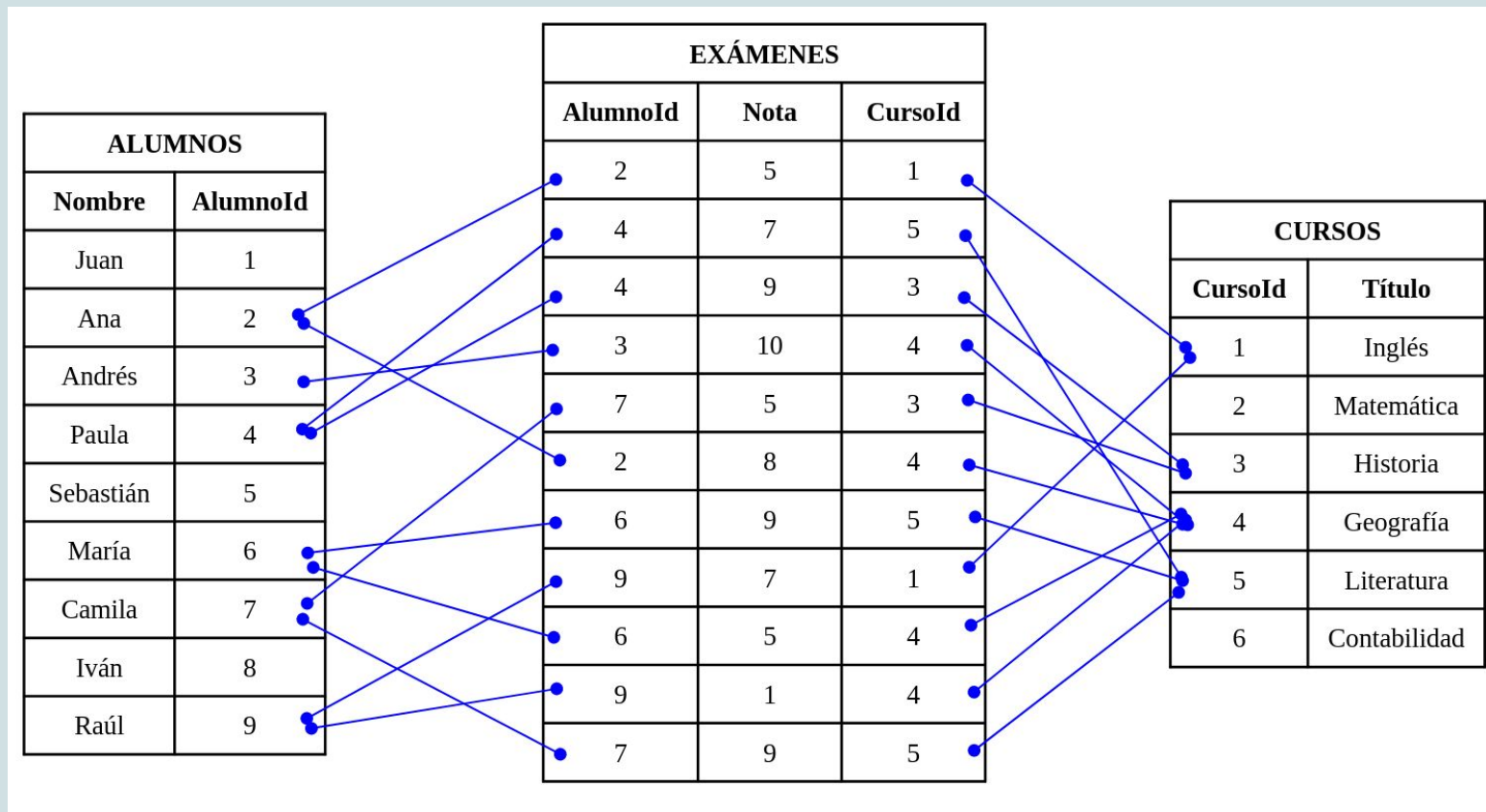
2) Listar por consola la cantidad de veces que se repiten los elementos de un vector de enteros. Ordenar por cantidad de repeticiones. Completar el siguiente código para que la salida por consola sea la indicada

```
int[] vector = [1, 3, 4, 5, 9, 4, 3, 4, 5, 1, 1, 4, 9, 4, 3, 1];  
vector.GroupBy(n => n)  
    // . . . completar aquí las líneas que faltan usando fluent API  
    .ForEach(obj => Console.WriteLine(obj));
```

Salida por consola

```
{ Numero = 5, Cantidad = 2 }  
{ Numero = 9, Cantidad = 2 }  
{ Numero = 3, Cantidad = 3 }  
{ Numero = 1, Cantidad = 4 }  
{ Numero = 4, Cantidad = 5 }
```

3) Supongamos que tenemos la siguiente información sobre alumnos, cursos y exámenes estructurada de esta manera:



Podemos ver, por ejemplo, que Ana sacó 5 en un examen de Inglés y 8 en un examen de Geografía, Juan no rindió ningún examen y nadie rindió examen en el curso de Matemáticas.

Definir las clases **Alumno**, **Examen** y **Curso**. Establecer por código las listas **alumnos** (de tipo **List<Alumno>**), **exámenes** (de tipo **List<Examen>**) y **cursos** (de tipo **List<Curso>**) con los datos que se muestran en la imagen anterior y resolver utilizando LINQ:

a) Obtener el listado con los nombres de los alumnos que rindieron al menos un examen, ordenado alfabéticamente (tip: puede utilizarse el método de extensión **Distinct()** para obtener una secuencia de elementos no repetidos). La salida debería ser:

Salida por consola

```
Ana  
Andrés  
Camila  
María  
Paula  
Raúl
```

b) Obtener el listado con los cursos donde se hayan rendido exámenes. Se debe listar el título del curso junto con la cantidad de exámenes. El listado debe ordenarse por cantidad de exámenes. La salida debería ser:

Salida por consola

```
{ Título = Inglés, Cantidad = 2 }  
{ Título = Historia, Cantidad = 2 }  
{ Título = Literatura, Cantidad = 3 }  
{ Título = Geografía, Cantidad = 4 }
```

c) Obtener el listado con los alumnos que hayan rendido al menos un examen informando el nombre del alumno, el título del curso y la nota del examen. La salida debería ser:

Salida por consola

```
{ Alumno = Ana, Curso = Inglés, Nota = 5 }  
{ Alumno = Ana, Curso = Geografía, Nota = 8 }  
{ Alumno = Andrés, Curso = Geografía, Nota = 10 }  
{ Alumno = Paula, Curso = Literatura, Nota = 7 }  
{ Alumno = Paula, Curso = Historia, Nota = 9 }  
{ Alumno = María, Curso = Literatura, Nota = 9 }  
{ Alumno = María, Curso = Geografía, Nota = 5 }  
{ Alumno = Camila, Curso = Historia, Nota = 5 }  
{ Alumno = Camila, Curso = Literatura, Nota = 9 }  
{ Alumno = Raúl, Curso = Inglés, Nota = 7 }  
{ Alumno = Raúl, Curso = Geografía, Nota = 1 }
```

d) Filtrar el listado del punto anterior para mostrar sólo los casos aprobados (nota ≥ 6).

e) Obtener el listado con los nombres de los alumnos que no han rendido ningún examen.

f) Obtener el listado de los alumnos que hayan rendido algún examen junto con el promedio de todos sus exámenes. La salida debería ser:

Salida por consola

```
{ Alumno = Ana, Promedio = 6,5 }  
{ Alumno = Andrés, Promedio = 10 }  
{ Alumno = Paula, Promedio = 8 }  
{ Alumno = María, Promedio = 7 }  
{ Alumno = Camila, Promedio = 7 }  
{ Alumno = Raúl, Promedio = 4 }
```

4) **Implementación de Repositorios con Entity Framework Core y SQLite:** Adaptar el proyecto de infraestructura del TP1 para implementar la persistencia en una base de datos SQLite. Reimplementar cada una de las clases de repositorio concretas. Realizar las operaciones traduciéndolas a operaciones de Entity Framework Core.